AGILEX SCOUT MINI 4WD MOBILE ROBOT

Complete ROS 2 Jazzy & Gazebo Harmonic Simulation Pipeline

Comprehensive Source Code Manual, Engineering Working Principles & Staff Evaluation Guide

---

Project Title:	AgileX Scout Mini 4-Wheel Skid-Steer AMR Simulation, Kinematics & Autonomous Navigation
Student / Candidate:	Sailakshmi B.T. — Department of Electronics and Communication Engineering
Target Evaluation:	Final Year Engineering Project Review / Faculty Viva Voce Examination
Operating System:	Ubuntu 24.04 LTS (Noble Numbat)
ROS Framework:	ROS 2 Jazzy Jalisco
Physics Simulation:	Gazebo Harmonic (gz-sim8) with ODE rigid contact physics
Sensor Integration:	360° 2D LiDAR (20Hz), 6-Axis IMU (100Hz), RealSense RGB-D Depth Camera (30Hz)
Navigation Stack:	Nav2 (MPPI Controller, NavFn Dijkstra Global Planner, AMCL Localization)
Sensor Fusion:	Extended Kalman Filter (robot_localization fusing wheel odom + IMU yaw rate)

Student Project Overview: This document was prepared as a technical defense manual to explain every single line of ROS 2 code written for the AgileX Scout Mini autonomous mobile robot project. The project takes a mechanical CAD model created in SolidWorks, converts it into an accurate URDF kinematics model, simulates realistic physical dynamics in Gazebo Harmonic, and implements state estimation, mapping, and multi-station autonomous patrol navigation in ROS 2 Jazzy. Below each code listing, the working principle is explained in clear, practical engineering terms, highlighting the exact mathematical formulas used and the practical challenges solved.

1. Project Background, Physical Robot & Architecture

What is this project about?
The objective of this project is to create an industrial-grade digital twin and autonomous navigation system for the **AgileX Scout Mini**—a rugged 4-wheel drive (4WD) mobile robot designed for factory automation and warehouse logistics. Unlike standard two-wheel differential drive robots (like the TurtleBot), the Scout Mini has four independently driven wheels without steering knuckles. It turns by driving the left and right wheels at different speeds, which causes the wheels to intentionally skid against the ground (skid-steer kinematics).

Why is this challenging?
1. *Wheel Slip:* Because the wheels skid, counting wheel encoder ticks gives very inaccurate odometry. During turns, the robot drifts by 30° to 40° from reality. We solved this by using an **Extended Kalman Filter (EKF)** that fuses the wheel speeds with a 100Hz 6-axis IMU gyroscope.
2. *Mesh Collisions:* When we first imported the SolidWorks CAD files into Gazebo, the triangular mesh facets on the tires snagged on the ground, causing the robot to bounce violently. We solved this by replacing mesh collisions with smooth analytical cylinders.
3. *Coordinate Conventions:* SolidWorks CAD had the robot facing along the +Y axis. ROS standards (REP-103) strictly require +X forward, +Y left. We corrected this using a 90° fixed rotation in the URDF base_footprint joint.
4. *Modern Software:* This project runs on the newest **ROS 2 Jazzy Jalisco** and **Gazebo Harmonic (Gz Sim 8)**, which uses the new ros_gz_bridge architecture instead of older Gazebo Classic plugins.

Master Inventory of All 14 ROS 2 Codes Used

#	File Name & Location	Role in the System	Primary Node / Tool	Lines
1	**patrol_mission.py** src/assem2_robot/scripts/patrol_mission.py	Autonomous Waypoint Mission	nav2_simple_commander	91
2	**simulation.launch.py** src/assem2_robot/launch/simulation.launch.py	Master System Bringup	ROS 2 Launch Engine	228
3	**navigation.launch.py** src/assem2_robot/launch/navigation.launch.py	Nav2 Autonomous Bringup	nav2_bringup	66
4	**slam.launch.py** src/assem2_robot/launch/slam.launch.py	2D LiDAR SLAM Mapping	slam_toolbox	42
5	**ekf.launch.py** src/assem2_robot/launch/ekf.launch.py	EKF State Estimation	robot_localization	41
6	**simulation.launch.py** launch/simulation.launch.py (Root)	Direct Simulator Spawner	ros_gz_sim create	119
7	**ekf.yaml** src/assem2_robot/config/ekf.yaml	Sensor Fusion Config	ekf_node (15 states)	51
8	**slam_toolbox.yaml** src/assem2_robot/config/slam_toolbox.yaml	Pose Graph SLAM Config	Ceres Optimizer	41
9	**nav2_params.yaml** src/assem2_robot/config/nav2_params.yaml	Nav2 Motion & Costmaps	AMCL + MPPI + NavFn	488
10	**Assem2.SLDASM.urdf** src/assem2_robot/urdf/Assem2.SLDASM.urdf	Robot URDF Model & Sensors	robot_state_publisher	541
11	**amr_world.sdf** src/assem2_robot/worlds/amr_world.sdf	10m x 10m Testing Arena	Gazebo Harmonic	233
12	**amr_world_map.yaml** src/assem2_robot/maps/amr_world_map.yaml	Occupancy Map Metadata	nav2_map_server	6
13	**CMakeLists.txt** src/assem2_robot/CMakeLists.txt	Build System Instructions	ament_cmake	27
14	**package.xml** src/assem2_robot/package.xml	Package Dependencies	ROS 2 Package Manifest	25

Code #1: patrol_mission.py

Relative Workspace Path: src/assem2_robot/scripts/patrol_mission.py

Role in System:	Autonomous Multi-Station Waypoint Patrol Mission
Primary Tool / Node:	nav2_simple_commander (BasicNavigator Action Client)
Topics & Signals:	Subscribes: /navigate_to_pose feedback | Publishes: Action Goals

Functional Purpose: patrol_mission.py

This is the high-level brain of the autonomous robot. It commands the Scout Mini to patrol five industrial workstations inside the arena (Docking Base -> Loading Zone -> Inspection -> Unloading -> Perimeter Checkpoint). It converts human-friendly angles into 3D quaternions, monitors live navigation progress in meters, pauses at each station to simulate work, and cleanly cancels goals if stopped.

Full Source Code: patrol_mission.py

```
#!/usr/bin/env python3

import time
import math
import rclpy
from rclpy.duration import Duration
from geometry_msgs.msg import PoseStamped
from nav2_simple_commander.robot_navigator import BasicNavigator, TaskResult


def create_pose(navigator, x, y, yaw_degrees):
    pose = PoseStamped()
    pose.header.frame_id = 'map'
    pose.header.stamp = navigator.get_clock().now().to_msg()
    pose.pose.position.x = float(x)
    pose.pose.position.y = float(y)
    pose.pose.position.z = 0.0

    # Convert yaw from degrees to quaternion
    yaw_rad = math.radians(yaw_degrees)
    pose.pose.orientation.z = math.sin(yaw_rad / 2.0)
    pose.pose.orientation.w = math.cos(yaw_rad / 2.0)
    return pose


def main():
    rclpy.init()
    navigator = BasicNavigator()

    print("\n" + "="*65)
    print("■ AgileX Scout Mini: Autonomous AMR Multi-Station Patrol Mission")
    print("="*65)

    # Wait until Nav2 is fully active
    print("[AMR PATROL] ■ Waiting for Nav2 stack to become active...")
    navigator.waitUntilNav2Active()
    print("[AMR PATROL] ■ Nav2 is ACTIVE and ready!\n")

    # Define Patrol Stations inside the 10m x 10m Arena
    stations = [
        {"name": "Station A (Docking / Charging Base)", "x": 0.0, "y": 0.0, "yaw": 0.0, "wait": 2.0},
        {"name": "Station B (Loading Zone)", "x": 1.5, "y": 1.5, "yaw": 90.0, "wait": 3.0},
        {"name": "Station C (Inspection Point)", "x": -1.5, "y": 1.5, "yaw": 180.0, "wait": 2.0},
        {"name": "Station D (Unloading Area)", "x": -1.5, "y": -1.5, "yaw": -90.0, "wait": 3.0},
        {"name": "Station E (Perimeter Checkpoint)", "x": 1.5, "y": -1.5, "yaw": 0.0, "wait": 2.0},
    ]

    patrol_count = 1

    try:
        while rclpy.ok():
            print(f"\n■ --- Starting Patrol Loop #{patrol_count} ---")

            for station in stations:
                print(f"\n■ [PATROL] Heading to {station['name']} -> ({station['x']:.1f}, {station['y']:.1f})...")
                goal_pose = create_pose(navigator, station['x'], station['y'], station['yaw'])

                navigator.goToPose(goal_pose)

                i = 0
                while not navigator.isTaskComplete():
                    feedback = navigator.getFeedback()
                    if feedback and i % 5 == 0:
                        distance_remaining = feedback.distance_remaining
                        print(f"   ■ Distance remaining: {distance_remaining:.2f} m | Navigating autonomously...")
                    time.sleep(0.5)
                    i += 1

                result = navigator.getResult()
                if result == TaskResult.SUCCEEDED:
                    print(f"   ■ Arrived at {station['name']}!")
                    print(f"   ■ Waiting {station['wait']:.0f} seconds (Simulating Operation)...")
```

```
                    time.sleep(station['wait'])
                elif result == TaskResult.CANCELED:
                    print(f"   ■■ Navigation to {station['name']} was canceled.")
                elif result == TaskResult.FAILED:
                    print(f"   ■ Navigation to {station['name']} failed! Attempting recovery to
    next station...")

            patrol_count += 1

    except KeyboardInterrupt:
        print("\n■ Patrol mission interrupted by user. Canceling active goal...")
        navigator.cancelTask()

    navigator.lifecycleShutdown()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

How the Code Works (Line-by-Line & Engineering Logic): patrol_mission.py

- **1. Imports & Navigator Setup (Lines 1-9):**

We import rclpy, geometry_msgs PoseStamped, and the official Nav2 Simple Commander API (BasicNavigator). Instead of manually writing complex ROS 2 Action Client callbacks, BasicNavigator gives us straightforward Python methods like goToPose() and isTaskComplete() to talk directly to Nav2's Behavior Tree.

- **2. Math Function: create_pose (Lines 11-23):**

Computers and ROS require 3D orientations as unit quaternions (x, y, z, w) rather than simple yaw angles in degrees. Our create_pose() helper takes an (x, y) coordinate and a yaw heading in degrees. It converts yaw to radians and computes: q_z = sin(yaw / 2) and q_w = cos(yaw / 2), while q_x and q_y are set to 0.0 since our robot stays on flat ground. This guarantees 100% compliance with ROS REP-103 coordinates without gimbal lock.

- **3. Lifecycle Synchronization Barrier (Lines 26-38):**

In main(), we call navigator.waitUntilNav2Active(). This is an essential safety check! In ROS 2, Nav2 nodes are Managed Lifecycle Nodes that start unconfigured. If you send navigation goals before the planner and controller reach the 'Active' state, the goals are rejected. This barrier waits until every Nav2 server is alive and ready.

- **4. Defining the 5 Patrol Stations (Lines 40-47):**

We configure five strategic stations inside our 10m x 10m warehouse arena: Station A at (0, 0) is the charging dock; Station B at (1.5, 1.5, 90°) is loading; Station C at (-1.5, 1.5, 180°) is inspection; Station D at (-1.5, -1.5, -90°) is unloading; and Station E at (1.5, -1.5, 0°) is the perimeter checkpoint. Each station has a dwell time (e.g. 3 seconds) where the robot pauses to simulate material transfer.

- **5. The Execution Loop & Feedback Telemetry (Lines 50-80):**

The while rclpy.ok() loop runs continuously. For each station, we build the PoseStamped and dispatch it with navigator.goToPose(goal_pose). An inner while loop checks navigator.isTaskComplete(). Every 2.5 seconds, it reads feedback.distance_remaining and prints live distance to the terminal. If the goal succeeds, the robot waits its dwell time; if an obstacle completely blocks the station and navigation fails, it catches the failure and moves to the next station.

- **6. Clean Emergency Stop (Lines 81-87):**

If the operator presses Ctrl+C, the try-except KeyboardInterrupt block catches it immediately. It calls navigator.cancelTask() to stop the motors, shuts down the lifecycle managers, and terminates cleanly.

Evaluator & Staff Viva Voce Q&A;

Q: Why did we use quaternions instead of yaw angles?

A: ROS 2 navigation actions require PoseStamped messages which use 3D unit quaternions. Using quaternions avoids singularities and gimbal lock.

Q: What happens if a dynamic obstacle blocks the path?

A: Nav2's Behavior Tree triggers recovery actions (spin or back up). If the path remains completely blocked, navigator.getResult() returns FAILED, and our script safely logs the error and moves on to the next patrol station.

Code #2: simulation.launch.py (assem2_robot)

Relative Workspace Path: src/assem2_robot/launch/simulation.launch.py

Role in System:	Master System Bringup & Inter-Process Orchestration
Primary Tool / Node:	ROS 2 Launch Engine (gz sim, robot_state_publisher, ros_gz_bridge, rviz2)
Topics & Signals:	Bridges 10 topics between Gazebo Harmonic and ROS 2 Jazzy

Functional Purpose: simulation.launch.py (assem2_robot)

This is the single master launch file that starts our entire project with one command. It isolates our network on localhost, sets mesh resource paths, launches Gazebo Harmonic with the 10m arena, runs robot_state_publisher to publish the TF tree, establishes the two-way ros_gz_bridge, opens RViz2, and conditionally includes EKF sensor fusion, SLAM mapping, or Nav2 autonomous navigation.

Full Source Code: simulation.launch.py (assem2_robot)

```
import os
from pathlib import Path

from ament_index_python.packages import get_package_share_directory, PackageNotFoundError
from launch import LaunchDescription
from launch.actions import (
    DeclareLaunchArgument,
    ExecuteProcess,
    IncludeLaunchDescription,
    SetEnvironmentVariable
)
from launch.conditions import IfCondition
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import EnvironmentVariable, LaunchConfiguration
from launch_ros.actions import Node


def generate_launch_description():
    pkg_name = 'assem2_robot'
    package_path = get_package_share_directory(pkg_name)
    package_share_path = os.path.dirname(package_path)

    urdf_path = os.path.join(package_path, 'urdf', 'Assem2.SLDASM.urdf')
    rviz_path = os.path.join(package_path, 'config', 'display.rviz')
    ekf_config_path = os.path.join(package_path, 'config', 'ekf.yaml')
    slam_config_path = os.path.join(package_path, 'config', 'slam_toolbox.yaml')
    nav2_config_path = os.path.join(package_path, 'config', 'nav2_params.yaml')
    default_map_path = os.path.join(package_path, 'maps', 'amr_world_map.yaml')

    with open(urdf_path, 'r') as f:
        robot_description_content = f.read()

    world_path = os.path.join(package_path, 'worlds', 'amr_world.sdf')

    use_sim_time = LaunchConfiguration('use_sim_time', default='true')
    use_ekf = LaunchConfiguration('use_ekf', default='true')
    use_slam = LaunchConfiguration('use_slam', default='false')
    use_nav = LaunchConfiguration('use_nav', default='false')
    map_file = LaunchConfiguration('map', default=default_map_path)
    world = LaunchConfiguration('world', default=world_path)

    declare_use_sim_time = DeclareLaunchArgument(
        'use_sim_time',
        default_value='true',
        description='Use simulation (Gazebo) clock if true'
    )

    declare_use_ekf = DeclareLaunchArgument(
        'use_ekf',
        default_value='true',
        description='Enable EKF sensor fusion (robot_localization)'
    )

    declare_use_slam = DeclareLaunchArgument(
        'use_slam',
        default_value='false',
        description='Enable SLAM mapping (slam_toolbox)'
    )

    declare_use_nav = DeclareLaunchArgument(
        'use_nav',
        default_value='false',
        description='Enable Nav2 autonomous navigation stack'
    )

    declare_map = DeclareLaunchArgument(
        'map',
        default_value=default_map_path,
        description='Full path to map YAML file for Nav2'
    )

    declare_world = DeclareLaunchArgument(
        'world',
        default_value=world_path,
        description='Path to Gazebo world SDF file'
    )

    # Direct Gazebo Harmonic execution
    gazebo = ExecuteProcess(
```

```
        cmd=['gz', 'sim', '-r', world_path],
        output='screen'
    )

    # Robot State Publisher
    robot_state_publisher = Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        name='robot_state_publisher',
        output='screen',
        parameters=[
            {'robot_description': robot_description_content},
            {'use_sim_time': use_sim_time}
        ]
    )

    # ROS-Gazebo Parameter Bridge for actuators, odometry, and integrated sensors
    bridge = Node(
        package='ros_gz_bridge',
        executable='parameter_bridge',
        output='screen',
        arguments=[
            # Actuation & Teleop
            '/cmd_vel@geometry_msgs/msg/Twist@gz.msgs.Twist',
            # Odometry & Joint States & Clock
            '/odom@nav_msgs/msg/Odometry[gz.msgs.Odometry',
            '/world/amr_world/clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock',
            '/joint_states@sensor_msgs/msg/JointState[gz.msgs.Model',
            # 2D LiDAR
            '/scan@sensor_msgs/msg/LaserScan[gz.msgs.LaserScan',
            # IMU
            '/imu@sensor_msgs/msg/Imu[gz.msgs.IMU',
            # RGB-D / Depth Camera
            '/camera/image@sensor_msgs/msg/Image[gz.msgs.Image',
            '/camera/camera_info@sensor_msgs/msg/CameraInfo[gz.msgs.CameraInfo',
            '/camera/depth_image@sensor_msgs/msg/Image[gz.msgs.Image',
            '/camera/points@sensor_msgs/msg/PointCloud2[gz.msgs.PointCloudPacked',
        ],
        remappings=[
            ('/world/amr_world/clock', '/clock'),
            ('/camera/image', '/camera/image_raw')
        ],
        parameters=[{'use_sim_time': use_sim_time}]
    )

    # RViz2 Node
    rviz = Node(
        package='rviz2',
        executable='rviz2',
        name='rviz2',
        output='screen',
        arguments=['-d', rviz_path] if os.path.exists(rviz_path) else []
    )

    nodes_to_launch = [
        SetEnvironmentVariable(name='ROS_AUTOMATIC_DISCOVERY_RANGE', value='LOCALHOST'),
        SetEnvironmentVariable(
            name='GZ_SIM_RESOURCE_PATH',
            value=[
                package_share_path,
                ':',
                EnvironmentVariable('GZ_SIM_RESOURCE_PATH', default_value='')
            ]
        ),
        SetEnvironmentVariable(
            name='IGN_GAZEBO_RESOURCE_PATH',
            value=[
                package_share_path,
                ':',
                EnvironmentVariable('IGN_GAZEBO_RESOURCE_PATH', default_value='')
            ]
        ),
        declare_use_sim_time,
        declare_use_ekf,
        declare_use_slam,
        declare_use_nav,
        declare_map,
        declare_world,
        gazebo,
        robot_state_publisher,
        bridge,
```

```
161 |         rviz
162 |     ]
163 |
164 |     try:
165 |         get_package_share_directory('robot_localization')
166 |         robot_localization_node = Node(
167 |             package='robot_localization',
168 |             executable='ekf_node',
169 |             name='ekf_filter_node',
170 |             output='screen',
171 |             parameters=[
172 |                 ekf_config_path,
173 |                 {'use_sim_time': use_sim_time}
174 |             ],
175 |             remappings=[
176 |                 ('odometry/filtered', '/odometry/filtered')
177 |             ],
178 |             condition=IfCondition(use_ekf)
179 |         )
180 |         nodes_to_launch.append(robot_localization_node)
181 |     except PackageNotFoundError:
182 |         pass
183 |
184 |     try:
185 |         get_package_share_directory('slam_toolbox')
186 |         slam_launch = IncludeLaunchDescription(
187 |             PythonLaunchDescriptionSource(
188 |                 os.path.join(
189 |                     get_package_share_directory('slam_toolbox'),
190 |                     'launch',
191 |                     'online_async_launch.py'
192 |                 )
193 |             ),
194 |             launch_arguments={
195 |                 'use_sim_time': use_sim_time,
196 |                 'slam_params_file': slam_config_path,
197 |                 'autostart': 'true'
198 |             }.items(),
199 |             condition=IfCondition(use_slam)
200 |         )
201 |         nodes_to_launch.append(slam_launch)
202 |     except PackageNotFoundError:
203 |         pass
204 |
205 |     try:
206 |         get_package_share_directory('nav2_bringup')
207 |         nav2_launch = IncludeLaunchDescription(
208 |             PythonLaunchDescriptionSource(
209 |                 os.path.join(
210 |                     get_package_share_directory('nav2_bringup'),
211 |                     'launch',
212 |                     'bringup_launch.py'
213 |                 )
214 |             ),
215 |             launch_arguments={
216 |                 'map': map_file,
217 |                 'use_sim_time': use_sim_time,
218 |                 'params_file': nav2_config_path,
219 |                 'autostart': 'true'
220 |             }.items(),
221 |             condition=IfCondition(use_nav)
222 |         )
223 |         nodes_to_launch.append(nav2_launch)
224 |     except PackageNotFoundError:
225 |         pass
226 |
227 |     return LaunchDescription(nodes_to_launch)
```

How the Code Works (Line-by-Line & Engineering Logic): simulation.launch.py (assem2_robot)

- **1. Environment Isolation (Lines 134-151):**

We set ROS_AUTOMATIC_DISCOVERY_RANGE=LOCALHOST. In college computer labs or shared WiFi, other students' ROS nodes can accidentally cross-talk and control your robot. Setting this variable restricts DDS communication to 127.0.0.1. We also export GZ_SIM_RESOURCE_PATH so Gazebo can locate the robot's STL mesh files.

- **2. Launch Arguments for Modular Operation (Lines 42-77):**

We declare launch arguments: use_sim_time (true), use_ekf (true), use_slam (default false), and use_nav (default false). This makes our project completely modular: for mapping we launch with use_slam:=true; for autonomous navigation we launch with use_nav:=true.

- **3. Direct Gazebo Harmonic Execution (Lines 79-83):**

Instead of using heavy wrappers, we use ExecuteProcess to run 'gz sim -r <world_path>' directly in simulation mode (-r), loading our 10m x 10m amr_world.sdf environment with deterministic 5ms ODE physics.

- **4. Robot State Publisher (Lines 84-94):**

Reads our Assem2.SLDASM.urdf and joint states from the simulator, calculating the 3D position of every wheel, sensor, and link, and publishing the full /tf coordinate frame tree.

- **5. Bidirectional ros_gz_bridge (Lines 96-123):**

Gazebo Harmonic uses Google Protobuf messages, while ROS 2 uses ROS message types. The parameter_bridge translates between them: /cmd_vel sends drive commands to Gazebo; /odom, /scan, /imu, /joint_states, and camera streams flow from Gazebo back to ROS. Crucially, /world/amr_world/clock is remapped to /clock so ROS simulation time stays perfectly synchronized with Gazebo physics.

- **6. Conditional Subsystem Ingestion (Lines 164-226):**

Using IfCondition, the launch file appends the robot_localization EKF node if use_ekf=true, includes slam_toolbox if use_slam=true, and includes nav2_bringup if use_nav=true. It uses try-except blocks so that missing optional packages won't crash the launch.

Evaluator & Staff Viva Voce Q&A;

Q: Why is remapping /world/amr_world/clock to /clock necessary?

A: Gazebo Harmonic outputs simulation time on a world-specific topic. ROS 2 nodes look for the standard global /clock topic when use_sim_time is true to keep timestamps aligned across all nodes.

Q: What does robot_state_publisher do?

A: It reads the kinematic joint positions from /joint_states and applies forward kinematics using the URDF model to broadcast the dynamic 3D transformation tree on /tf.

Code #3: navigation.launch.py

Relative Workspace Path: src/assem2_robot/launch/navigation.launch.py

Role in System:	Nav2 Autonomous Navigation Bringup
Primary Tool / Node:	nav2_bringup (AMCL, Costmaps, NavfnPlanner, MPPIController, BT Navigator)
Topics & Signals:	Publishes: /cmd_vel, /plan, costmaps | Subscribes: /scan, /odometry/filtered, /tf

Functional Purpose: navigation.launch.py

Dedicated launch file that boots up the entire Navigation 2 stack. It binds our pre-built 2D occupancy grid arena map (amr_world_map.yaml) and our tuned parameter configuration (nav2_params.yaml), launching AMCL particle filter localization, dynamic costmaps, global Dijkstra path planning, and MPPI trajectory tracking.

Full Source Code: navigation.launch.py

```
 1 | import os
 2 | from ament_index_python.packages import get_package_share_directory
 3 | from launch import LaunchDescription
 4 | from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription
 5 | from launch.launch_description_sources import PythonLaunchDescriptionSource
 6 | from launch.substitutions import LaunchConfiguration
 7 |
 8 |
 9 | def generate_launch_description():
10 |     pkg_name = 'assem2_robot'
11 |     package_path = get_package_share_directory(pkg_name)
12 |     nav2_bringup_dir = get_package_share_directory('nav2_bringup')
13 |
14 |     default_map_path = os.path.join(package_path, 'maps', 'amr_world_map.yaml')
15 |     default_params_path = os.path.join(package_path, 'config', 'nav2_params.yaml')
16 |
17 |     map_yaml_file = LaunchConfiguration('map')
18 |     params_file = LaunchConfiguration('params_file')
19 |     use_sim_time = LaunchConfiguration('use_sim_time')
20 |     autostart = LaunchConfiguration('autostart')
21 |
22 |     declare_map_yaml_cmd = DeclareLaunchArgument(
23 |         'map',
24 |         default_value=default_map_path,
25 |         description='Full path to map yaml file to load'
26 |     )
27 |
28 |     declare_params_file_cmd = DeclareLaunchArgument(
29 |         'params_file',
30 |         default_value=default_params_path,
31 |         description='Full path to the ROS2 parameters file to use for all launched nodes'
32 |     )
33 |
34 |     declare_use_sim_time_cmd = DeclareLaunchArgument(
35 |         'use_sim_time',
36 |         default_value='true',
37 |         description='Use simulation (Gazebo) clock if true'
38 |     )
39 |
40 |     declare_autostart_cmd = DeclareLaunchArgument(
41 |         'autostart',
42 |         default_value='true',
43 |         description='Automatically startup the nav2 stack'
44 |     )
45 |
46 |     # Nav2 Bringup Launch
47 |     nav2_bringup_launch = IncludeLaunchDescription(
48 |         PythonLaunchDescriptionSource(
49 |             os.path.join(nav2_bringup_dir, 'launch', 'bringup_launch.py')
50 |         ),
51 |         launch_arguments={
52 |             'map': map_yaml_file,
53 |             'use_sim_time': use_sim_time,
54 |             'params_file': params_file,
55 |             'autostart': autostart
56 |         }.items()
57 |     )
58 |
59 |     return LaunchDescription([
60 |         declare_map_yaml_cmd,
61 |         declare_params_file_cmd,
62 |         declare_use_sim_time_cmd,
63 |         declare_autostart_cmd,
64 |         nav2_bringup_launch
65 |     ])
```

How the Code Works (Line-by-Line & Engineering Logic): navigation.launch.py

- **1. Path Resolution & Defaults (Lines 10-21):**

Finds the installed share directories for assem2_robot and nav2_bringup using ament_index_python. It sets the default map path to amr_world_map.yaml and parameters to nav2_params.yaml.

- **2. Argument Declarations (Lines 22-45):**

Exposes 'map', 'params_file', 'use_sim_time', and 'autostart' as configurable CLI options. Setting autostart:=true tells the Nav2 Lifecycle Manager to automatically transition every node into the active state.

- **3. Including nav2_bringup/bringup_launch.py (Lines 46-58):**

Invokes the official Nav2 bringup pipeline with our parameters. This starts the Map Server, AMCL, Planner Server, Controller Server, Smoother Server, Behavior Tree Navigator, and Waypoint Follower simultaneously.

Evaluator & Staff Viva Voce Q&A;

Q: How do we swap the map in this launch file?

A: By passing the map argument at the terminal: ros2 launch assem2_robot navigation.launch.py map:=/path/to/my_map.yaml.

Q: What does autostart: true do?

A: It commands the nav2_lifecycle_manager to automatically step all navigation nodes through Unconfigured -> Inactive -> Active states without needing manual service triggers.

Code #4: slam.launch.py

Relative Workspace Path: src/assem2_robot/launch/slam.launch.py

Role in System:	2D Pose-Graph SLAM Mapping Bringup
Primary Tool / Node:	slam_toolbox (async_slam_toolbox_node)
Topics & Signals:	Subscribes: /scan, /tf (odom -> base_footprint) | Publishes: /map, /tf (map -> odom)

Functional Purpose: slam.launch.py

Dedicated launch file for building a 2D occupancy grid map of an unknown environment. It feeds the robot's 2D LiDAR range scans and wheel odometry into the SLAM Toolbox async node, which runs Ceres non-linear least squares optimization to correct odometry drift and generate the /map topic.

Full Source Code: slam.launch.py

```
 1 | import os
 2 | from ament_index_python.packages import get_package_share_directory
 3 | from launch import LaunchDescription
 4 | from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription
 5 | from launch.launch_description_sources import PythonLaunchDescriptionSource
 6 | from launch.substitutions import LaunchConfiguration
 7 |
 8 |
 9 | def generate_launch_description():
10 |     pkg_name = 'assem2_robot'
11 |     package_path = get_package_share_directory(pkg_name)
12 |     slam_config_path = os.path.join(package_path, 'config', 'slam_toolbox.yaml')
13 |
14 |     use_sim_time = LaunchConfiguration('use_sim_time', default='true')
15 |
16 |     declare_use_sim_time = DeclareLaunchArgument(
17 |         'use_sim_time',
18 |         default_value='true',
19 |         description='Use simulation (Gazebo) clock if true'
20 |     )
21 |
22 |     # Launch slam_toolbox with auto-transition lifecycle management
23 |     slam_launch = IncludeLaunchDescription(
24 |         PythonLaunchDescriptionSource(
25 |             os.path.join(
26 |                 get_package_share_directory('slam_toolbox'),
27 |                 'launch',
28 |                 'online_async_launch.py'
29 |             )
30 |         ),
31 |         launch_arguments={
32 |             'use_sim_time': use_sim_time,
33 |             'slam_params_file': slam_config_path,
34 |             'autostart': 'true'
35 |         }.items()
36 |     )
37 |
38 |     return LaunchDescription([
39 |         declare_use_sim_time,
40 |         slam_launch
41 |     ])
```

How the Code Works (Line-by-Line & Engineering Logic): slam.launch.py

- **1. Parameter Loading (Lines 10-21):**

Resolves config/slam_toolbox.yaml and exposes use_sim_time defaulting to true.

- **2. Asynchronous Launch Include (Lines 22-36):**

Includes 'online_async_launch.py' from slam_toolbox. Asynchronous mapping ensures that heavy submap matching runs in a background thread, preventing sensor drops or lag during fast robot motion.

- **3. Coordinate Frame Correction:**

As SLAM runs, it calculates the cumulative drift of the robot's wheels and continuously broadcasts the map -> odom transform, ensuring that the map stays perfectly stable in RViz2 while the robot navigates.

Evaluator & Staff Viva Voce Q&A;

Q: Why use online_async instead of online_sync in SLAM Toolbox?

A: In online_async mode, scan matching runs on worker threads. If a loop closure takes longer to solve, incoming laser scans are queued rather than dropped, keeping the system responsive in real time.

Code #5: ekf.launch.py

Relative Workspace Path: src/assem2_robot/launch/ekf.launch.py

Role in System:	Standalone Extended Kalman Filter State Estimation
Primary Tool / Node:	robot_localization (ekf_node)
Topics & Signals:	Subscribes: /odom, /imu | Publishes: /odometry/filtered, /tf (odom -> base_footprint)

Functional Purpose: ekf.launch.py

Brings up the 15-state Extended Kalman Filter from the robot_localization package. Fuses the noisy, slip-prone wheel encoder odometry (/odom) with the high-frequency angular rate from the 6-axis IMU gyroscope (/imu), broadcasting the filtered odometry on /odometry/filtered and the TF tree.

Full Source Code: ekf.launch.py

```
 1 | import os
 2 | from ament_index_python.packages import get_package_share_directory
 3 | from launch import LaunchDescription
 4 | from launch.actions import DeclareLaunchArgument
 5 | from launch.substitutions import LaunchConfiguration
 6 | from launch_ros.actions import Node
 7 |
 8 |
 9 | def generate_launch_description():
10 |     pkg_name = 'assem2_robot'
11 |     package_path = get_package_share_directory(pkg_name)
12 |     ekf_config_path = os.path.join(package_path, 'config', 'ekf.yaml')
13 |
14 |     use_sim_time = LaunchConfiguration('use_sim_time', default='true')
15 |
16 |     declare_use_sim_time = DeclareLaunchArgument(
17 |         'use_sim_time',
18 |         default_value='true',
19 |         description='Use simulation (Gazebo) clock if true'
20 |     )
21 |
22 |     # EKF Filter Node from robot_localization
23 |     ekf_node = Node(
24 |         package='robot_localization',
25 |         executable='ekf_node',
26 |         name='ekf_filter_node',
27 |         output='screen',
28 |         parameters=[
29 |             ekf_config_path,
30 |             {'use_sim_time': use_sim_time}
31 |         ],
32 |         remappings=[
33 |             ('odometry/filtered', '/odometry/filtered')
34 |         ]
35 |     )
36 |
37 |     return LaunchDescription([
38 |         declare_use_sim_time,
39 |         ekf_node
40 |     ])
```

How the Code Works (Line-by-Line & Engineering Logic): ekf.launch.py

- **1. Node Definition & Remapping (Lines 22-35):**

Spawns 'ekf_node' with our ekf.yaml configuration. Remaps internal odometry/filtered to the standard root topic /odometry/filtered.

- **2. Eliminating Heading Drift:**

In a 4-wheel skid-steer robot, tires slip when turning. Wheel odometry alone reports wildly inaccurate headings. This launch file starts the EKF filter that uses the IMU's rate gyroscope to anchor the robot's heading.

Evaluator & Staff Viva Voce Q&A;

Q: What frame transform does this node publish?

A: It publishes the dynamic continuous odom -> base_footprint transform, representing the robot's estimated position and heading relative to its starting point.

Code #6: simulation.launch.py (Root)

Relative Workspace Path: launch/simulation.launch.py

Role in System:	Direct Robot Spawner in Empty Simulation
Primary Tool / Node:	ros_gz_sim (gz_sim.launch.py, create spawner node), robot_state_publisher
Topics & Signals:	Bridges /cmd_vel, /odom, /tf, /clock, /joint_states

Functional Purpose: simulation.launch.py (Root)

A lightweight, standalone bringup file located at the workspace root. It boots up Gazebo Harmonic with an empty world, reads the raw URDF string, spawns the robot at coordinates (0, 0, 0.02m) using the ros_gz_sim 'create' node, and starts RViz2.

Full Source Code: simulation.launch.py (Root)

```
import os
from pathlib import Path

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription, SetEnvironmentVaria
  ble
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import EnvironmentVariable, LaunchConfiguration
from launch_ros.actions import Node


def generate_launch_description():
    pkg_name = 'assem2_robot'
    package_path = get_package_share_directory(pkg_name)
    package_share_path = os.path.dirname(package_path)

    urdf_path = os.path.join(package_path, 'urdf', 'Assem2.SLDASM.urdf')
    rviz_path = os.path.join(package_path, 'config', 'display.rviz')

    with open(urdf_path, 'r') as f:
        robot_description_content = f.read()

    use_sim_time = LaunchConfiguration('use_sim_time', default='true')

    declare_use_sim_time = DeclareLaunchArgument(
        'use_sim_time',
        default_value='true',
        description='Use simulation (Gazebo) clock if true'
    )

    # Gazebo Sim launch (Gazebo Harmonic / Gz Sim)
    gazebo = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(
                get_package_share_directory('ros_gz_sim'),
                'launch',
                'gz_sim.launch.py'
            )
        ),
        launch_arguments={'gz_args': '-r empty.sdf'}.items()
    )

    # Robot State Publisher
    robot_state_publisher = Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        name='robot_state_publisher',
        output='screen',
        parameters=[
            {'robot_description': robot_description_content},
            {'use_sim_time': use_sim_time}
        ]
    )

    # Spawn Robot in Gazebo (single instance, no auto-duplication)
    spawn_robot = Node(
        package='ros_gz_sim',
        executable='create',
        output='screen',
        arguments=[
            '-name', 'assem2_robot',
            '-string', robot_description_content,
            '-allow_renaming', 'false',
            '-x', '0.0',
            '-y', '0.0',
            '-z', '0.02'
        ]
    )

    # ROS-Gazebo Parameter Bridge
    bridge = Node(
        package='ros_gz_bridge',
        executable='parameter_bridge',
        output='screen',
        arguments=[
            '/cmd_vel@geometry_msgs/msg/Twist@gz.msgs.Twist',
            '/odom@nav_msgs/msg/Odometry[gz.msgs.Odometry',
            '/tf@tf2_msgs/msg/TFMessage[gz.msgs.Pose_V',
```

```
            '/clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock',
            '/joint_states@sensor_msgs/msg/JointState[gz.msgs.Model',
        ],
        parameters=[{'use_sim_time': use_sim_time}]
    )

    # RViz2 Node
    rviz = Node(
        package='rviz2',
        executable='rviz2',
        name='rviz2',
        output='screen',
        arguments=['-d', rviz_path] if os.path.exists(rviz_path) else [],
        parameters=[{'use_sim_time': use_sim_time}]
    )

    return LaunchDescription([
        SetEnvironmentVariable(
            name='GZ_SIM_RESOURCE_PATH',
            value=[
                package_share_path,
                ':',
                EnvironmentVariable('GZ_SIM_RESOURCE_PATH', default_value='')
            ]
        ),
        SetEnvironmentVariable(
            name='IGN_GAZEBO_RESOURCE_PATH',
            value=[
                package_share_path,
                ':',
                EnvironmentVariable('IGN_GAZEBO_RESOURCE_PATH', default_value='')
            ]
        ),
        declare_use_sim_time,
        gazebo,
        robot_state_publisher,
        spawn_robot,
        bridge,
        rviz
    ])
```

How the Code Works (Line-by-Line & Engineering Logic): simulation.launch.py (Root)

- **1. Model Spawner Node (Lines 55-68):**

Invokes 'ros_gz_sim/create' with '-string robot_description_content', injecting the robot model directly from system memory without requiring an intermediate temporary file on disk.

- **2. Essential Bridge Setup (Lines 70-84):**

Bridges velocity commands, raw odometry, clock, and joint states for rapid testing of robot kinematics.

Evaluator & Staff Viva Voce Q&A;

Q: When do you use this root launch file versus the package simulation launch file?

A: This file is used for quick kinematic testing in an empty world without obstacles. The package simulation.launch.py is our complete arena bringup with full sensors, EKF, and Nav2.

Code #7: ekf.yaml

Relative Workspace Path: src/assem2_robot/config/ekf.yaml

Role in System:	Extended Kalman Filter 15-State Fusion Configuration
Primary Tool / Node:	ekf_filter_node (robot_localization)
Topics & Signals:	Inputs: /odom, /imu | Output: /odometry/filtered

Functional Purpose: ekf.yaml

Configures the mathematical matrix weights for the 15-state Extended Kalman Filter. Enforces 2D planar constraints, fuses wheel encoder position and velocities, and fuses the IMU gyroscope yaw rate to produce drift-free state estimation.

Full Source Code: ekf.yaml

```
### EKF Sensor Fusion Configuration for AgileX Scout Mini AMR
### Fuses Wheel Odometry (/odom) + 6-Axis IMU (/imu) into /odometry/filtered
ekf_filter_node:
  ros__parameters:
    use_sim_time: true
    frequency: 30.0
    sensor_timeout: 0.1
    two_d_mode: true
    transform_time_offset: 0.0
    transform_timeout: 0.0
    print_diagnostics: true
    debug: false

    # Coordinate Frames
    map_frame: map
    odom_frame: odom
    base_link_frame: base_footprint
    world_frame: odom

    # 1. Wheel Odometry Input (/odom from DiffDrive)
    # [x, y, z, roll, pitch, yaw, vx, vy, vz, vroll, vpitch, vyaw, ax, ay, az]
    odom0: /odom
    odom0_config: [true,  true,  false,
                   false, false, true,
                   true,  false, false,
                   false, false, true,
                   false, false, false]
    odom0_queue_size: 10
    odom0_nodelay: true
    odom0_differential: false
    odom0_relative: false

    # 2. 6-Axis IMU Input (/imu from IMU sensor)
    # [x, y, z, roll, pitch, yaw, vx, vy, vz, vroll, vpitch, vyaw, ax, ay, az]
    imu0: /imu
    imu0_config: [false, false, false,
                  false, false, false,
                  false, false, false,
                  false, false, true,
                  false, false, false]
    imu0_queue_size: 10
    imu0_nodelay: true
    imu0_differential: false
    imu0_relative: false
    imu0_remove_gravitational_acceleration: true

    # Outputs
    use_control: false
    publish_tf: true
    publish_acceleration: false
```

How the Code Works (Line-by-Line & Engineering Logic): ekf.yaml

- **1. General Filter Parameters (Lines 1-13):**

frequency: 30.0 Hz (predict-update loop rate). two_d_mode: true is critical: it forces z, roll, pitch, and their velocities to 0.0, preventing vertical bounce or phantom roll during turns on flat factory floors.

- **2. Coordinate Frames (Lines 14-19):**

world_frame is set to 'odom'. This keeps the EKF operating in a continuous, local odometric frame without abrupt discrete jumps, leaving global map corrections to AMCL or SLAM.

- **3. Wheel Odometry Fusion Mask: odom0 (Lines 20-32):**

The 15-element boolean mask represents [X, Y, Z, Roll, Pitch, Yaw, Vx, Vy, Vz, Vroll, Vpitch, Vyaw, Ax, Ay, Az]. We set True for X, Y, Yaw, Vx, and Vyaw. We reject Z, roll, pitch, and lateral velocity Vy (since wheels don't slide freely sideways).

- **4. IMU Fusion Mask: imu0 (Lines 33-46):**

For the IMU, we set True ONLY for the 12th element: Yaw velocity (Vyaw / angular velocity around Z). Why? Double-integrating noisy accelerometer data causes exponential position drift. Fusing only the rate gyro gives the EKF an exact measurement of turning speed, directly correcting wheel slip errors.

Evaluator & Staff Viva Voce Q&A;

Q: What happens if you fuse IMU linear accelerations directly?

A: Consumer IMU accelerometers have small DC bias errors. Integrating acceleration twice to get position causes drift that grows quadratically with time (t^2), rapidly corrupting the robot's position.

Code #8: slam_toolbox.yaml

Relative Workspace Path: src/assem2_robot/config/slam_toolbox.yaml

Role in System:	SLAM Toolbox Graph Optimization & Scan Matching Parameters
Primary Tool / Node:	slam_toolbox (Ceres Solver)
Topics & Signals:	Input: /scan, /tf | Output: /map, /tf (map -> odom)

Functional Purpose: slam_toolbox.yaml

Configures the Google Ceres non-linear least squares solver and correlative scan matcher for 2D SLAM. Sets laser range to 12.0m, map resolution to 5cm/pixel, and defines angular search tolerances for loop closures.

Full Source Code: slam_toolbox.yaml

```
slam_toolbox:
  ros__parameters:
    # Solver Plugin
    solver_plugin: solver_plugins::CeresSolver
    ceres_linear_solver: SPARSE_NORMAL_CHOLESKY
    ceres_preconditioner: SCHUR_JACOBI
    ceres_trust_strategy: LEVENBERG_MARQUARDT
    ceres_dogleg_type: TRADITIONAL_DOGLEG
    ceres_loss_function: None

    # ROS / Coordinate Frames
    odom_frame: odom
    map_frame: map
    base_frame: base_footprint
    scan_topic: /scan
    use_map_saver: true
    mode: mapping

    # Tuning Parameters
    debug_logging: false
    throttle_scans: 1
    transform_publish_period: 0.02 # 50hz
    map_update_interval: 1.0
    resolution: 0.05
    max_laser_range: 12.0
    minimum_time_interval: 0.05
    transform_timeout: 0.2
    tf_buffer_duration: 30.0
    stack_size_to_use: 40000000
    enable_interactive_mode: true

    # Scan Matching
    distance_variance_penalty: 0.5
    angle_variance_penalty: 1.0
    fine_search_angle_offset: 0.00349
    coarse_search_angle_offset: 0.349
    coarse_angle_resolution: 0.0349
    minimum_angle_penalty: 0.9
    minimum_distance_penalty: 0.5
    use_response_expansion: true
```

How the Code Works (Line-by-Line & Engineering Logic): slam_toolbox.yaml

- **1. Ceres Optimization Engine (Lines 1-10):**

Configures solver_plugins::CeresSolver using SPARSE_NORMAL_CHOLESKY linear solver, SCHUR_JACOBI preconditioner, and LEVENBERG_MARQUARDT trust-region strategy. This mathematically optimizes pose constraints across the entire trajectory graph.

- **2. Map Resolution & Laser Range (Lines 22-26):**

resolution: 0.05 (each grid pixel represents exactly 5cm x 5cm). max_laser_range: 12.0m matches our physical LiDAR specifications.

- **3. Scan Matching Search Windows (Lines 32-41):**

coarse_search_angle_offset: 0.349 rad (~20°) and fine_search_angle_offset: 0.00349 rad (~0.2°). When the robot turns quickly, the correlative scan matcher searches a 20° window to align point clouds, then refines it to sub-degree accuracy.

Evaluator & Staff Viva Voce Q&A;

Q: What does the resolution: 0.05 parameter mean?

A: It means each cell in the occupancy grid map represents a 5cm by 5cm physical square in the real world.

Code #9: nav2_params.yaml

Relative Workspace Path: src/assem2_robot/config/nav2_params.yaml

Role in System:	Complete Nav2 Navigation Stack Configuration
Primary Tool / Node:	AMCL, NavFn Dijkstra Planner, MPPI Local Controller, Costmaps, Behavior Server
Topics & Signals:	Inputs: /scan, /camera/points, /odometry/filtered | Output: /cmd_vel

Functional Purpose: nav2_params.yaml

The most comprehensive configuration in the project (488 lines). Configures AMCL particle filtering, global and local costmaps (with 2D LiDAR and 3D depth camera voxel clearing), Dijkstra shortest-path planning, and the Model Predictive Path Integral (MPPI) local trajectory controller.

Full Source Code: nav2_params.yaml

```yaml
amcl:
  ros__parameters:
    set_initial_pose: true
    initial_pose:
      x: 0.0
      y: 0.0
      z: 0.0
      yaw: 0.0
    alpha1: 0.2
    alpha2: 0.2
    alpha3: 0.2
    alpha4: 0.2
    alpha5: 0.2
    base_frame_id: "base_footprint"
    beam_skip_distance: 0.5
    beam_skip_error_threshold: 0.9
    beam_skip_threshold: 0.3
    do_beamskip: false
    global_frame_id: "map"
    lambda_short: 0.1
    laser_likelihood_max_dist: 2.0
    laser_max_range: 12.0
    laser_min_range: 0.1
    laser_model_type: "likelihood_field"
    max_beams: 180
    max_particles: 2000
    min_particles: 500
    odom_frame_id: "odom"
    pf_err: 0.05
    pf_z: 0.99
    recovery_alpha_fast: 0.0
    recovery_alpha_slow: 0.0
    resample_interval: 1
    robot_model_type: "nav2_amcl::DifferentialMotionModel"
    save_pose_rate: 0.5
    sigma_hit: 0.2
    tf_broadcast: true
    transform_tolerance: 1.0
    update_min_a: 0.1
    update_min_d: 0.1
    z_hit: 0.8
    z_max: 0.05
    z_rand: 0.05
    z_short: 0.1
    scan_topic: /scan

bt_navigator:
  ros__parameters:
    global_frame: map
    robot_base_frame: base_footprint
    odom_topic: /odometry/filtered
    bt_loop_duration: 10
    default_server_timeout: 20
    wait_for_service_timeout: 1000
    action_server_result_timeout: 900.0
    navigators: ["navigate_to_pose", "navigate_through_poses"]
    navigate_to_pose:
      plugin: "nav2_bt_navigator::NavigateToPoseNavigator"
    navigate_through_poses:
      plugin: "nav2_bt_navigator::NavigateThroughPosesNavigator"
    # 'default_nav_through_poses_bt_xml' and 'default_nav_to_pose_bt_xml' are use defaults:
    # nav2_bt_navigator/navigate_to_pose_w_replanning_and_recovery.xml
    # nav2_bt_navigator/navigate_through_poses_w_replanning_and_recovery.xml
    # They can be set here or via a RewrittenYaml remap from a parent launch file to Nav2.

    # plugin_lib_names is used to add custom BT plugins to the executor (vector of strings).
    # Built-in plugins are added automatically
    # plugin_lib_names: []

    error_code_names:
      - compute_path_error_code
      - follow_path_error_code

controller_server:
  ros__parameters:
    controller_frequency: 20.0
    costmap_update_timeout: 0.30
    min_x_velocity_threshold: 0.001
    min_y_velocity_threshold: 0.5
```

```
    min_theta_velocity_threshold: 0.001
    failure_tolerance: 0.3
    progress_checker_plugins: ["progress_checker"]
    goal_checker_plugins: ["general_goal_checker"] # "precise_goal_checker"
    controller_plugins: ["FollowPath"]
    use_realtime_priority: false

    # Progress checker parameters
    progress_checker:
      plugin: "nav2_controller::SimpleProgressChecker"
      required_movement_radius: 0.5
      movement_time_allowance: 10.0
    # Goal checker parameters
    #precise_goal_checker:
    #  plugin: "nav2_controller::SimpleGoalChecker"
    #  xy_goal_tolerance: 0.25
    #  yaw_goal_tolerance: 0.25
    #  stateful: True
    general_goal_checker:
      stateful: True
      plugin: "nav2_controller::SimpleGoalChecker"
      xy_goal_tolerance: 0.25
      yaw_goal_tolerance: 0.25
    FollowPath:
      plugin: "nav2_mppi_controller::MPPIController"
      time_steps: 56
      model_dt: 0.05
      batch_size: 2000
      ax_max: 3.0
      ax_min: -3.0
      ay_max: 3.0
      ay_min: -3.0
      az_max: 3.5
      vx_std: 0.2
      vy_std: 0.2
      wz_std: 0.4
      vx_max: 0.5
      vx_min: -0.35
      vy_max: 0.5
      wz_max: 1.9
      iteration_count: 1
      prune_distance: 1.7
      transform_tolerance: 0.1
      temperature: 0.3
      gamma: 0.015
      motion_model: "DiffDrive"
      visualize: true
      regenerate_noises: true
      TrajectoryVisualizer:
        trajectory_step: 5
        time_step: 3
      AckermannConstraints:
        min_turning_r: 0.2
      critics: [
        "ConstraintCritic", "CostCritic", "GoalCritic",
        "GoalAngleCritic", "PathAlignCritic", "PathFollowCritic",
        "PathAngleCritic", "PreferForwardCritic"]
      ConstraintCritic:
        enabled: true
        cost_power: 1
        cost_weight: 4.0
      GoalCritic:
        enabled: true
        cost_power: 1
        cost_weight: 5.0
        threshold_to_consider: 1.4
      GoalAngleCritic:
        enabled: true
        cost_power: 1
        cost_weight: 3.0
        threshold_to_consider: 0.5
      PreferForwardCritic:
        enabled: true
        cost_power: 1
        cost_weight: 5.0
        threshold_to_consider: 0.5
      CostCritic:
        enabled: true
        cost_power: 1
        cost_weight: 3.81
        near_collision_cost: 253
```

```
        critical_cost: 300.0
        consider_footprint: false
        collision_cost: 1000000.0
        near_goal_distance: 1.0
        trajectory_point_step: 2
      PathAlignCritic:
        enabled: true
        cost_power: 1
        cost_weight: 14.0
        max_path_occupancy_ratio: 0.05
        trajectory_point_step: 4
        threshold_to_consider: 0.5
        offset_from_furthest: 20
        use_path_orientations: false
      PathFollowCritic:
        enabled: true
        cost_power: 1
        cost_weight: 5.0
        offset_from_furthest: 5
        threshold_to_consider: 1.4
      PathAngleCritic:
        enabled: true
        cost_power: 1
        cost_weight: 2.0
        offset_from_furthest: 4
        threshold_to_consider: 0.5
        max_angle_to_furthest: 1.0
        mode: 0
      # TwirlingCritic:
      #   enabled: true
      #   twirling_cost_power: 1
      #   twirling_cost_weight: 10.0

local_costmap:
  local_costmap:
    ros__parameters:
      update_frequency: 5.0
      publish_frequency: 2.0
      global_frame: odom
      robot_base_frame: base_footprint
      rolling_window: true
      width: 4
      height: 4
      resolution: 0.05
      footprint: "[ [0.35, 0.30], [0.35, -0.30], [-0.35, -0.30], [-0.35, 0.30] ]"
      plugins: ["voxel_layer", "inflation_layer"]
      inflation_layer:
        plugin: "nav2_costmap_2d::InflationLayer"
        cost_scaling_factor: 3.0
        inflation_radius: 0.55
      voxel_layer:
        plugin: "nav2_costmap_2d::VoxelLayer"
        enabled: True
        publish_voxel_map: True
        origin_z: 0.0
        z_resolution: 0.05
        z_voxels: 16
        max_obstacle_height: 2.0
        mark_threshold: 0
        observation_sources: scan pointcloud
        scan:
          topic: /scan
          max_obstacle_height: 2.0
          clearing: True
          marking: True
          data_type: "LaserScan"
          raytrace_max_range: 12.0
          raytrace_min_range: 0.0
          obstacle_max_range: 10.0
          obstacle_min_range: 0.0
        pointcloud:
          topic: /camera/points
          max_obstacle_height: 1.8
          min_obstacle_height: 0.04
          clearing: True
          marking: True
          data_type: "PointCloud2"
          raytrace_max_range: 6.0
          raytrace_min_range: 0.2
          obstacle_max_range: 5.0
          obstacle_min_range: 0.2
```

```
      static_layer:
        plugin: "nav2_costmap_2d::StaticLayer"
        map_subscribe_transient_local: True
      always_send_full_costmap: True

global_costmap:
  global_costmap:
    ros__parameters:
      update_frequency: 1.0
      publish_frequency: 1.0
      global_frame: map
      robot_base_frame: base_footprint
      footprint: "[ [0.35, 0.30], [0.35, -0.30], [-0.35, -0.30], [-0.35, 0.30] ]"
      resolution: 0.05
      track_unknown_space: true
      plugins: ["static_layer", "obstacle_layer", "inflation_layer"]
      obstacle_layer:
        plugin: "nav2_costmap_2d::ObstacleLayer"
        enabled: True
        observation_sources: scan
        scan:
          topic: /scan
          max_obstacle_height: 2.0
          clearing: True
          marking: True
          data_type: "LaserScan"
          raytrace_max_range: 12.0
          raytrace_min_range: 0.0
          obstacle_max_range: 10.0
          obstacle_min_range: 0.0
      static_layer:
        plugin: "nav2_costmap_2d::StaticLayer"
        map_subscribe_transient_local: True
      inflation_layer:
        plugin: "nav2_costmap_2d::InflationLayer"
        cost_scaling_factor: 3.0
        inflation_radius: 0.7
      always_send_full_costmap: True

# The yaml_filename does not need to be specified since it going to be set by defaults in launch.
# If you'd rather set it in the yaml, remove the default "map" value in the tb3_simulation_launch.py
# file & provide full path to map below. If CLI map configuration or launch default is provided, that will be used.
# map_server:
#   ros__parameters:
#     yaml_filename: ""

map_saver:
  ros__parameters:
    save_map_timeout: 5.0
    free_thresh_default: 0.25
    occupied_thresh_default: 0.65
    map_subscribe_transient_local: True

planner_server:
  ros__parameters:
    expected_planner_frequency: 20.0
    planner_plugins: ["GridBased"]
    costmap_update_timeout: 1.0
    GridBased:
      plugin: "nav2_navfn_planner::NavfnPlanner"
      tolerance: 0.5
      use_astar: false
      allow_unknown: true

smoother_server:
  ros__parameters:
    smoother_plugins: ["simple_smoother"]
    simple_smoother:
      plugin: "nav2_smoother::SimpleSmoother"
      tolerance: 1.0e-10
      max_its: 1000
      do_refinement: True

behavior_server:
  ros__parameters:
    local_costmap_topic: local_costmap/costmap_raw
    global_costmap_topic: global_costmap/costmap_raw
    local_footprint_topic: local_costmap/published_footprint
```

```
    global_footprint_topic: global_costmap/published_footprint
    cycle_frequency: 10.0
    behavior_plugins: ["spin", "backup", "drive_on_heading", "assisted_teleop", "wait"]
    spin:
      plugin: "nav2_behaviors::Spin"
    backup:
      plugin: "nav2_behaviors::BackUp"
    drive_on_heading:
      plugin: "nav2_behaviors::DriveOnHeading"
    wait:
      plugin: "nav2_behaviors::Wait"
    assisted_teleop:
      plugin: "nav2_behaviors::AssistedTeleop"
    local_frame: odom
    global_frame: map
    robot_base_frame: base_footprint
    transform_tolerance: 0.1
    simulate_ahead_time: 2.0
    max_rotational_vel: 1.0
    min_rotational_vel: 0.4
    rotational_acc_lim: 3.2

waypoint_follower:
  ros__parameters:
    loop_rate: 20
    stop_on_failure: false
    action_server_result_timeout: 900.0
    waypoint_task_executor_plugin: "wait_at_waypoint"
    wait_at_waypoint:
      plugin: "nav2_waypoint_follower::WaitAtWaypoint"
      enabled: True
      waypoint_pause_duration: 200

route_server:
  ros__parameters:
    # The graph_filepath does not need to be specified since it going to be set by defaults in
  launch.
    # If you'd rather set it in the yaml, remove the default "graph" value in the launch file(s
  ).
    # file & provide full path to map below. If graph config or launch default is provided, it
  is used
    # graph_filepath: $(find-pkg-share nav2_route)/graphs/aws_graph.geojson
    boundary_radius_to_achieve_node: 1.0
    radius_to_achieve_node: 2.0
    smooth_corners: true
    operations: ["AdjustSpeedLimit", "ReroutingService", "CollisionMonitor"]
    ReroutingService:
      plugin: "nav2_route::ReroutingService"
    AdjustSpeedLimit:
      plugin: "nav2_route::AdjustSpeedLimit"
    CollisionMonitor:
      plugin: "nav2_route::CollisionMonitor"
      max_collision_dist: 3.0
    edge_cost_functions: ["DistanceScorer", "CostmapScorer"]
    DistanceScorer:
      plugin: "nav2_route::DistanceScorer"
    CostmapScorer:
      plugin: "nav2_route::CostmapScorer"

velocity_smoother:
  ros__parameters:
    smoothing_frequency: 20.0
    stamp_smoothed_velocity_with_smoothing_time: False
    scale_velocities: False
    feedback: "OPEN_LOOP"
    max_velocity: [0.5, 0.0, 2.0]
    min_velocity: [-0.5, 0.0, -2.0]
    max_accel: [2.5, 0.0, 3.2]
    max_decel: [-2.5, 0.0, -3.2]
    odom_topic: "odom"
    odom_duration: 0.1
    deadband_velocity: [0.0, 0.0, 0.0]
    velocity_timeout: 1.0

collision_monitor:
  ros__parameters:
    base_frame_id: "base_footprint"
    odom_frame_id: "odom"
    cmd_vel_in_topic: "cmd_vel_smoothed"
    cmd_vel_out_topic: "cmd_vel"
    state_topic: "collision_monitor_state"
```

```
    transform_tolerance: 0.2
    source_timeout: 1.0
    base_shift_correction: True
    stop_pub_timeout: 2.0
    # Polygons represent zone around the robot for "stop", "slowdown" and "limit" action types,
    # and robot footprint for "approach" action type.
    polygons: ["FootprintApproach"]
    FootprintApproach:
      type: "polygon"
      action_type: "approach"
      footprint_topic: "/local_costmap/published_footprint"
      time_before_collision: 1.2
      simulation_time_step: 0.1
      min_points: 6
      visualize: False
      enabled: True
    observation_sources: ["scan"]
    scan:
      type: "scan"
      topic: "scan"
      min_height: 0.15
      max_height: 2.0
      enabled: True

docking_server:
  ros__parameters:
    controller_frequency: 50.0
    initial_perception_timeout: 5.0
    wait_charge_timeout: 5.0
    dock_approach_timeout: 30.0
    undock_linear_tolerance: 0.05
    undock_angular_tolerance: 0.1
    max_retries: 3
    base_frame: "base_footprint"
    fixed_frame: "odom"
    dock_backwards: false
    dock_prestaging_tolerance: 0.5

    # Types of docks
    dock_plugins: ['simple_charging_dock']
    simple_charging_dock:
      plugin: 'opennav_docking::SimpleChargingDock'
      docking_threshold: 0.05
      staging_x_offset: -0.7
      use_external_detection_pose: true
      use_battery_status: false # true
      use_stall_detection: false # true

      external_detection_timeout: 1.0
      external_detection_translation_x: -0.18
      external_detection_translation_y: 0.0
      external_detection_rotation_roll: -1.57
      external_detection_rotation_pitch: -1.57
      external_detection_rotation_yaw: 0.0
      filter_coef: 0.1

    # Dock instances
    # The following example illustrates configuring dock instances.
    # docks: ['home_dock']  # Input your docks here
    # home_dock:
    #   type: 'simple_charging_dock'
    #   frame: map
    #   pose: [0.0, 0.0, 0.0]

    controller:
      k_phi: 3.0
      k_delta: 2.0
      v_linear_min: 0.15
      v_linear_max: 0.15
      use_collision_detection: true
      costmap_topic: "local_costmap/costmap_raw"
      footprint_topic: "local_costmap/published_footprint"
      transform_tolerance: 0.1
      projection_time: 5.0
      simulation_step: 0.1
      dock_collision_threshold: 0.3

loopback_simulator:
  ros__parameters:
    base_frame_id: "base_footprint"
    odom_frame_id: "odom"
```

```
479 |     map_frame_id: "map"
480 |     scan_frame_id: "base_scan"  # tb4_loopback_simulator.launch.py remaps to 'rplidar_link'
481 |     update_duration: 0.02
482 |     scan_range_min: 0.05
483 |     scan_range_max: 30.0
484 |     scan_angle_min: -3.1415
485 |     scan_angle_max: 3.1415
486 |     scan_angle_increment: 0.02617
487 |     scan_use_inf: true
```

How the Code Works (Line-by-Line & Engineering Logic): nav2_params.yaml

- **1. AMCL Particle Filter Localization (Lines 1-46):**

Maintains 500 to 2000 random particle hypotheses. Uses a 'likelihood_field' sensor model that compares real-time LiDAR scans against the static arena map. DifferentialMotionModel noise parameters alpha1-alpha5 are tuned to 0.2.

- **2. Global & Local Costmaps (Lines 194-280):**

Defines the robot footprint as a rectangular box [[0.35, 0.30], [0.35, -0.30], [-0.35, -0.30], [-0.35, 0.30]] (70cm x 60cm). The local costmap uses a 4m x 4m rolling window centered on base_footprint. It includes a voxel_layer that marks obstacles from both 2D LiDAR (/scan) and 3D depth camera point clouds (/camera/points up to 1.8m height). An inflation_layer adds exponential cost gradients (radius 0.55m-0.70m, scaling factor 3.0) to keep the robot centered away from walls.

- **3. Global Planner: NavfnPlanner (Lines 295-305):**

Implements Dijkstra's shortest-path dynamic programming algorithm across the 2D global grid costmap to find the optimal path.

- **4. Local Controller: MPPIController (Lines 104-193):**

Model Predictive Path Integral controller running at 20Hz. Samples 2000 candidate trajectories (batch_size: 2000) over a 56-step prediction horizon (dt=0.05s). Evaluates rollouts using weighted critics: PathAlignCritic (weight 14.0) keeps it on the path, CostCritic (weight 3.81) avoids obstacles, and GoalCritic (weight 5.0) guides it to the waypoint.

- **5. Velocity Smoother & Recovery (Lines 315-390):**

Clamps linear acceleration to 2.5 m/s^2 and angular acceleration to 3.2 rad/s^2 to prevent jerky motor torque demands. Configures recovery behaviors (Spin, BackUp, DriveOnHeading, Wait) if the robot gets boxed in by obstacles.

Evaluator & Staff Viva Voce Q&A;

Q: Why did you choose MPPI over the standard DWB (Dynamic Window Approach)?

A: MPPI samples thousands of dynamic trajectory rollouts in parallel using GPU/CPU vectorization. It handles non-linear skid-steer dynamics and tight cornering around obstacles far better than DWB, which often gets stuck in local minima.

Code #10: Assem2.SLDASM.urdf

Relative Workspace Path: src/assem2_robot/urdf/Assem2.SLDASM.urdf

Role in System:	Robot Mechanical Kinematics, Dynamics & Sensor Plugins
Primary Tool / Node:	robot_state_publisher, gz-sim (DiffDrive, JointStatePublisher, Sensors)
Topics & Signals:	Outputs: /odom, /scan, /imu, /camera/image, /camera/points, /joint_states

Functional Purpose: Assem2.SLDASM.urdf

The master mechanical and sensor description of the AgileX Scout Mini. Exported from SolidWorks and calibrated for ROS 2: aligns CAD coordinates with ROS REP-103 (+X forward), defines masses and low CoM, replaces mesh collisions with smooth cylinders, sets skid-steer friction parameters, and attaches Gazebo Harmonic DiffDrive, 2D LiDAR, 6-axis IMU, and RGB-D camera plugins.

Full Source Code: Assem2.SLDASM.urdf

```
<?xml version="1.0" encoding="utf-8"?>
<robot name="assem2_robot">

  <!-- Base Footprint (Ground Projection Root Link) -->
  <link name="base_footprint" />

  <!-- Fixed joint aligning physical CAD chassis front with ROS REP-103 (+X is Forward, +Y is Left) -->
  <joint name="base_footprint_joint" type="fixed">
    <parent link="base_footprint" />
    <child link="base_link" />
    <origin xyz="-0.10185 0.16662 0.2269" rpy="0 0 1.5707963" />
  </joint>

  <!-- Base Link (Chassis) -->
  <link name="base_link">
    <inertial>
      <!-- Low Center of Mass located at bottom chassis (Z ~ 0.10m in footprint) for high roll/pitch stability -->
      <origin xyz="-0.16662 -0.10185 -0.12" rpy="0 0 0" />
      <mass value="22.0" />
      <inertia ixx="0.35" ixy="0.0" ixz="0.0" iyy="0.45" iyz="0.0" izz="0.65" />
    </inertial>
    <visual>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <geometry>
        <mesh filename="package://assem2_robot/meshes/base_link.STL" />
      </geometry>
      <material name="metallic_blue">
        <color rgba="0.2 0.3 0.6 1.0" />
      </material>
    </visual>
    <collision>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <geometry>
        <mesh filename="package://assem2_robot/meshes/base_link.STL" />
      </geometry>
    </collision>
  </link>

  <!-- Wheel 1: Front Left in physical ROS heading (w1) -->
  <link name="w1">
    <inertial>
      <origin xyz="0.062 0 0" rpy="0 0 0" />
      <mass value="1.5" />
      <inertia ixx="0.005" ixy="0" ixz="0" iyy="0.003" iyz="0" izz="0.003" />
    </inertial>
    <visual>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <geometry>
        <mesh filename="package://assem2_robot/meshes/w1.STL" />
      </geometry>
      <material name="dark_rubber">
        <color rgba="0.15 0.15 0.15 1.0" />
      </material>
    </visual>
    <!-- Smooth analytical cylinder collision prevents mesh-vertex snagging against obstacles -->
    <collision>
      <origin xyz="0.062 0 0" rpy="0 1.5707963 0" />
      <geometry>
        <cylinder radius="0.08" length="0.06" />
      </geometry>
    </collision>
  </link>

  <joint name="j1" type="continuous">
    <origin xyz="0.13938 -0.29727 -0.14629" rpy="2.33386 0 0" />
    <parent link="base_link" />
    <child link="w1" />
    <axis xyz="1 0 0" />
    <dynamics damping="0.1" friction="0.1" />
    <limit effort="40" velocity="25" />
  </joint>

  <!-- Wheel 2: Rear Left in physical ROS heading (w2) -->
  <link name="w2">
    <inertial>
      <origin xyz="0.062 0 0" rpy="0 0 0" />
```

```
      <mass value="1.5" />
      <inertia ixx="0.005" ixy="0" ixz="0" iyy="0.003" iyz="0" izz="0.003" />
    </inertial>
    <visual>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <geometry>
        <mesh filename="package://assem2_robot/meshes/w2.STL" />
      </geometry>
      <material name="dark_rubber">
        <color rgba="0.15 0.15 0.15 1.0" />
      </material>
    </visual>
    <collision>
      <origin xyz="0.062 0 0" rpy="0 1.5707963 0" />
      <geometry>
        <cylinder radius="0.08" length="0.06" />
      </geometry>
    </collision>
  </link>

  <joint name="j2" type="continuous">
    <origin xyz="0.13938 0.09368 -0.14742" rpy="2.56241 0 0" />
    <parent link="base_link" />
    <child link="w2" />
    <axis xyz="1 0 0" />
    <dynamics damping="0.1" friction="0.1" />
    <limit effort="40" velocity="25" />
  </joint>

  <!-- Wheel 3: Front Right in physical ROS heading (w3) -->
  <link name="w3">
    <inertial>
      <origin xyz="0.062 0 0" rpy="0 0 0" />
      <mass value="1.5" />
      <inertia ixx="0.005" ixy="0" ixz="0" iyy="0.003" iyz="0" izz="0.003" />
    </inertial>
    <visual>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <geometry>
        <mesh filename="package://assem2_robot/meshes/w3.STL" />
      </geometry>
      <material name="dark_rubber">
        <color rgba="0.15 0.15 0.15 1.0" />
      </material>
    </visual>
    <collision>
      <origin xyz="0.062 0 0" rpy="0 1.5707963 0" />
      <geometry>
        <cylinder radius="0.08" length="0.06" />
      </geometry>
    </collision>
  </link>

  <joint name="j3" type="continuous">
    <origin xyz="-0.47262 -0.29727 -0.14629" rpy="-2.87166 0 3.14159" />
    <parent link="base_link" />
    <child link="w3" />
    <axis xyz="-1 0 0" />
    <dynamics damping="0.1" friction="0.1" />
    <limit effort="40" velocity="25" />
  </joint>

  <!-- Wheel 4: Rear Right in physical ROS heading (w4) -->
  <link name="w4">
    <inertial>
      <origin xyz="0.062 0 0" rpy="0 0 0" />
      <mass value="1.5" />
      <inertia ixx="0.005" ixy="0" ixz="0" iyy="0.003" iyz="0" izz="0.003" />
    </inertial>
    <visual>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <geometry>
        <mesh filename="package://assem2_robot/meshes/w4.STL" />
      </geometry>
      <material name="dark_rubber">
        <color rgba="0.15 0.15 0.15 1.0" />
      </material>
    </visual>
    <collision>
      <origin xyz="0.062 0 0" rpy="0 1.5707963 0" />
      <geometry>
```

```
158 |         <cylinder radius="0.08" length="0.06" />
159 |       </geometry>
160 |     </collision>
161 |   </link>
162 |
163 |   <joint name="j4" type="continuous">
164 |     <origin xyz="-0.47262 0.09348 -0.14780" rpy="-1.29916 0 3.14159" />
165 |     <parent link="base_link" />
166 |     <child link="w4" />
167 |     <axis xyz="-1 0 0" />
168 |     <dynamics damping="0.1" friction="0.1" />
169 |     <limit effort="40" velocity="25" />
170 |   </joint>
171 |
172 |   <!-- Gazebo Simulation Plugins (Gazebo Harmonic / Gz Sim) -->
173 |   <gazebo>
174 |     <!-- DiffDrive plugin for 4-wheel differential/skid-steer drive -->
175 |     <plugin filename="gz-sim-diff-drive-system" name="gz::sim::systems::DiffDrive">
176 |       <!-- Left side wheels (j1=Front Left, j2=Rear Left) -->
177 |       <left_joint>j1</left_joint>
178 |       <left_joint>j2</left_joint>
179 |       <!-- Right side wheels (j3=Front Right, j4=Rear Right) -->
180 |       <right_joint>j3</right_joint>
181 |       <right_joint>j4</right_joint>
182 |       <wheel_separation>0.612</wheel_separation>
183 |       <wheel_radius>0.08</wheel_radius>
184 |       <max_linear_acceleration>2.5</max_linear_acceleration>
185 |       <min_linear_acceleration>-2.5</min_linear_acceleration>
186 |       <max_angular_acceleration>4.0</max_angular_acceleration>
187 |       <min_angular_acceleration>-4.0</min_angular_acceleration>
188 |       <max_linear_velocity>2.0</max_linear_velocity>
189 |       <min_linear_velocity>-2.0</min_linear_velocity>
190 |       <max_angular_velocity>3.0</max_angular_velocity>
191 |       <min_angular_velocity>-3.0</min_angular_velocity>
192 |       <topic>/cmd_vel</topic>
193 |       <odom_topic>/odom</odom_topic>
194 |       <tf_topic>/tf</tf_topic>
195 |       <frame_id>odom</frame_id>
196 |       <child_frame_id>base_footprint</child_frame_id>
197 |       <odom_publish_frequency>50</odom_publish_frequency>
198 |       <cmd_vel_timeout>0.5</cmd_vel_timeout>
199 |     </plugin>
200 |
201 |     <!-- Joint state publisher plugin -->
202 |     <plugin filename="gz-sim-joint-state-publisher-system" name="gz::sim::systems::JointStatePu
    |   blisher">
203 |       <topic>/joint_states</topic>
204 |       <update_rate>50</update_rate>
205 |       <joint_name>j1</joint_name>
206 |       <joint_name>j2</joint_name>
207 |       <joint_name>j3</joint_name>
208 |       <joint_name>j4</joint_name>
209 |     </plugin>
210 |   </gazebo>
211 |
212 |   <!-- Wheel Surface & Contact Friction in Gazebo -->
213 |   <gazebo reference="w1">
214 |     <mu1>1.0</mu1>
215 |     <mu2>0.1</mu2>
216 |     <kp>1000000.0</kp>
217 |     <kd>100.0</kd>
218 |     <minDepth>0.001</minDepth>
219 |   </gazebo>
220 |   <gazebo reference="w2">
221 |     <mu1>1.0</mu1>
222 |     <mu2>0.1</mu2>
223 |     <kp>1000000.0</kp>
224 |     <kd>100.0</kd>
225 |     <minDepth>0.001</minDepth>
226 |   </gazebo>
227 |   <gazebo reference="w3">
228 |     <mu1>1.0</mu1>
229 |     <mu2>0.1</mu2>
230 |     <kp>1000000.0</kp>
231 |     <kd>100.0</kd>
232 |     <minDepth>0.001</minDepth>
233 |   </gazebo>
234 |   <gazebo reference="w4">
235 |     <mu1>1.0</mu1>
236 |     <mu2>0.1</mu2>
237 |     <kp>1000000.0</kp>
```

```
    <kd>100.0</kd>
    <minDepth>0.001</minDepth>
  </gazebo>

  <!-- ================================================================= -->
  <!-- SENSORS ATTACHMENTS & GAZEBO HARMONIC SENSOR CONFIGURATIONS       -->
  <!-- ================================================================= -->

  <!-- 1. 2D LiDAR Sensor (LaserScan) -->
  <link name="lidar_mount_link">
    <inertial>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <mass value="0.08" />
      <inertia ixx="0.00005" ixy="0" ixz="0" iyy="0.00005" iyz="0" izz="0.00005" />
    </inertial>
    <visual>
      <origin xyz="0 0 0.015" rpy="0 0 0" />
      <geometry>
        <cylinder radius="0.045" length="0.03" />
      </geometry>
      <material name="mount_metal">
        <color rgba="0.25 0.25 0.25 1.0" />
      </material>
    </visual>
    <collision>
      <origin xyz="0 0 0.015" rpy="0 0 0" />
      <geometry>
        <cylinder radius="0.045" length="0.03" />
      </geometry>
    </collision>
  </link>

  <joint name="lidar_mount_joint" type="fixed">
    <parent link="base_link" />
    <child link="lidar_mount_link" />
    <!-- Top center deck of Scout Mini (X=0, Y=0 in footprint, Z~0.33m) -->
    <origin xyz="-0.16662 -0.10185 0.105" rpy="0 0 -1.5707963" />
  </joint>

  <link name="lidar_link">
    <inertial>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <mass value="0.125" />
      <inertia ixx="0.0001" ixy="0" ixz="0" iyy="0.0001" iyz="0" izz="0.0001" />
    </inertial>
    <visual>
      <!-- Base cylinder -->
      <origin xyz="0 0 0.01" rpy="0 0 0" />
      <geometry>
        <cylinder radius="0.038" length="0.02" />
      </geometry>
      <material name="lidar_dark">
        <color rgba="0.12 0.12 0.15 1.0" />
      </material>
    </visual>
    <visual>
      <!-- Top rotating optical turret -->
      <origin xyz="0 0 0.025" rpy="0 0 0" />
      <geometry>
        <cylinder radius="0.035" length="0.015" />
      </geometry>
      <material name="lidar_optic">
        <color rgba="0.05 0.05 0.08 0.95" />
      </material>
    </visual>
    <collision>
      <origin xyz="0 0 0.02" rpy="0 0 0" />
      <geometry>
        <cylinder radius="0.04" length="0.04" />
      </geometry>
    </collision>
  </link>

  <joint name="lidar_joint" type="fixed">
    <parent link="lidar_mount_link" />
    <child link="lidar_link" />
    <!-- Elevated optical plane (Z ~ 0.38m from ground) with unobstructed 360-deg FOV -->
    <origin xyz="0 0 0.03" rpy="0 0 0" />
  </joint>

  <gazebo reference="lidar_link">
```

```
    <sensor name="lidar" type="gpu_lidar">
      <pose relative_to="lidar_link">0 0 0.02 0 0 0</pose>
      <always_on>true</always_on>
      <visualize>true</visualize>
      <update_rate>20</update_rate>
      <topic>/scan</topic>
      <gz_frame_id>lidar_link</gz_frame_id>
      <lidar>
        <scan>
          <horizontal>
            <samples>720</samples>
            <resolution>1</resolution>
            <min_angle>-3.14159265</min_angle>
            <max_angle>3.14159265</max_angle>
          </horizontal>
        </scan>
        <range>
          <min>0.12</min>
          <max>16.0</max>
          <resolution>0.01</resolution>
        </range>
        <noise>
          <type>gaussian</type>
          <mean>0.0</mean>
          <stddev>0.01</stddev>
        </noise>
      </lidar>
    </sensor>
  </gazebo>

  <!-- 2. IMU Sensor (Inertial Measurement Unit) -->
  <link name="imu_link">
    <inertial>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <mass value="0.01" />
      <inertia ixx="0.00001" ixy="0" ixz="0" iyy="0.00001" iyz="0" izz="0.00001" />
    </inertial>
    <visual>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <geometry>
        <box size="0.025 0.025 0.01" />
      </geometry>
      <material name="imu_orange">
        <color rgba="0.95 0.45 0.1 1.0" />
      </material>
    </visual>
  </link>

  <joint name="imu_joint" type="fixed">
    <parent link="base_link" />
    <child link="imu_link" />
    <!-- Centered near the robot Center of Mass (CoM) for minimal lever-arm error -->
    <origin xyz="-0.16662 -0.10185 0.02" rpy="0 0 -1.5707963" />
  </joint>

  <gazebo reference="imu_link">
    <sensor name="imu_sensor" type="imu">
      <pose relative_to="imu_link">0 0 0 0 0 0</pose>
      <always_on>true</always_on>
      <update_rate>100</update_rate>
      <visualize>true</visualize>
      <topic>/imu</topic>
      <gz_frame_id>imu_link</gz_frame_id>
      <imu>
        <angular_velocity>
          <x>
            <noise type="gaussian">
              <mean>0.0</mean>
              <stddev>0.0002</stddev>
            </noise>
          </x>
          <y>
            <noise type="gaussian">
              <mean>0.0</mean>
              <stddev>0.0002</stddev>
            </noise>
          </y>
          <z>
            <noise type="gaussian">
              <mean>0.0</mean>
              <stddev>0.0002</stddev>
```

```
            </noise>
          </z>
        </angular_velocity>
        <linear_acceleration>
          <x>
            <noise type="gaussian">
              <mean>0.0</mean>
              <stddev>0.017</stddev>
            </noise>
          </x>
          <y>
            <noise type="gaussian">
              <mean>0.0</mean>
              <stddev>0.017</stddev>
            </noise>
          </y>
          <z>
            <noise type="gaussian">
              <mean>0.0</mean>
              <stddev>0.017</stddev>
            </noise>
          </z>
        </linear_acceleration>
      </imu>
    </sensor>
  </gazebo>

  <!-- 3. RGB-D / Depth Camera (RealSense Style Forward Perception) -->
  <link name="camera_mount_link">
    <inertial>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <mass value="0.04" />
      <inertia ixx="0.00002" ixy="0" ixz="0" iyy="0.00002" iyz="0" izz="0.00002" />
    </inertial>
    <visual>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <geometry>
        <box size="0.03 0.08 0.01" />
      </geometry>
      <material name="bracket_metal">
        <color rgba="0.3 0.3 0.35 1.0" />
      </material>
    </visual>
  </link>

  <joint name="camera_mount_joint" type="fixed">
    <parent link="base_link" />
    <child link="camera_mount_link" />
    <!-- Physical front bumper deck mounting location (+X forward in footprint) -->
    <origin xyz="-0.16662 -0.355 0.065" rpy="0 0 -1.5707963" />
  </joint>

  <link name="camera_link">
    <inertial>
      <origin xyz="0 0 0" rpy="0 0 0" />
      <mass value="0.07" />
      <inertia ixx="0.00005" ixy="0" ixz="0" iyy="0.00005" iyz="0" izz="0.00005" />
    </inertial>
    <!-- Camera main body -->
    <visual>
      <origin xyz="0 0 0.0125" rpy="0 0 0" />
      <geometry>
        <box size="0.025 0.09 0.025" />
      </geometry>
      <material name="camera_dark">
        <color rgba="0.1 0.1 0.1 1.0" />
      </material>
    </visual>
    <!-- Left Optical Lens -->
    <visual>
      <origin xyz="0.013 -0.025 0.0125" rpy="0 1.5707963 0" />
      <geometry>
        <cylinder radius="0.006" length="0.003" />
      </geometry>
      <material name="lens_glass">
        <color rgba="0.0 0.5 0.8 0.9" />
      </material>
    </visual>
    <!-- Right Optical Lens -->
    <visual>
      <origin xyz="0.013 0.025 0.0125" rpy="0 1.5707963 0" />
```

```
      <geometry>
        <cylinder radius="0.006" length="0.003" />
      </geometry>
      <material name="lens_glass">
        <color rgba="0.0 0.5 0.8 0.9" />
      </material>
    </visual>
    <collision>
      <origin xyz="0 0 0.0125" rpy="0 0 0" />
      <geometry>
        <box size="0.025 0.09 0.025" />
      </geometry>
    </collision>
  </link>

  <joint name="camera_joint" type="fixed">
    <parent link="camera_mount_link" />
    <child link="camera_link" />
    <!-- Front chassis mount facing forward (+X in ROS base_footprint) -->
    <origin xyz="0 0 0.01" rpy="0 0 0" />
  </joint>

  <!-- Standard ROS Optical Coordinate Frame (Z-forward, X-right, Y-down) -->
  <link name="camera_optical_frame" />

  <joint name="camera_optical_joint" type="fixed">
    <parent link="camera_link" />
    <child link="camera_optical_frame" />
    <origin xyz="0.013 0 0.0125" rpy="-1.5707963 0 -1.5707963" />
  </joint>

  <gazebo reference="camera_link">
    <sensor name="camera" type="rgbd_camera">
      <pose relative_to="camera_link">0.013 0 0.0125 0 0 0</pose>
      <always_on>true</always_on>
      <visualize>true</visualize>
      <update_rate>30</update_rate>
      <topic>/camera</topic>
      <gz_frame_id>camera_optical_frame</gz_frame_id>
      <camera>
        <horizontal_fov>1.3962634</horizontal_fov>
        <image>
          <width>640</width>
          <height>480</height>
          <format>R8G8B8</format>
        </image>
        <clip>
          <near>0.1</near>
          <far>10.0</far>
        </clip>
        <depth_camera>
          <clip>
            <near>0.1</near>
            <far>10.0</far>
          </clip>
        </depth_camera>
      </camera>
    </sensor>
  </gazebo>

</robot>
```

How the Code Works (Line-by-Line & Engineering Logic): Assem2.SLDASM.urdf

- **1. REP-103 Alignment (Lines 1-13):**

SolidWorks CAD modeled the robot with +Y as the front heading. ROS standard REP-103 mandates +X as forward and +Y as left. We created a fixed base_footprint_joint with yaw rotation rpy='0 0 1.5707963' (+90°) and spatial offsets (x=-0.10185, y=0.16662, z=0.2269m), aligning the physical CAD chassis front with the ROS forward driving vector.

- **2. Chassis Mass & Low Center of Mass (Lines 15-37):**

Chassis mass: 22.0 kg; wheels: 4 x 1.5 kg (total ~28 kg). The Center of Mass (origin Z = -0.12m in base_link, ~0.10m above ground) is kept very low, preventing the robot from tipping during fast zero-radius turns.

- **3. Analytical Cylinder Wheel Collisions (Lines 39-170):**

Raw STL mesh collisions have discrete triangular vertices that dig into the Gazebo ground plane. We replaced the collision geometry of all four wheels (w1, w2, w3, w4) with smooth analytical cylinders (r=0.08m, length=0.06m), yielding continuous contact physics without simulation jitter.

- **4. Gazebo DiffDrive Plugin (Lines 173-210):**

Plugin 'gz::sim::systems::DiffDrive' groups j1 (front-left) and j2 (rear-left) as left joints, and j3 (front-right) and j4 (rear-right) as right joints. Wheel track is 0.612m, wheel radius is 0.08m. It subscribes to /cmd_vel and calculates wheel angular speeds: w_L = (v - omega * b / 2) / r and w_R = (v + omega * b / 2) / r.

- **5. Anisotropic Skid-Steer Friction (Lines 212-241):**

For all 4 wheels, we set longitudinal friction mu1=1.0 (strong forward traction) and lateral friction mu2=0.1 (low sideways resistance). This allows the wheels to skid sideways smoothly when turning in place without stalling the motors.

- **6. Integrated Sensor Suite (Lines 246-539):**

LiDAR: gpu_lidar on lidar_link (Z ~ 0.38m) with 720 samples across 360°, 20Hz, 16m range, publishing to /scan. IMU: 100Hz on imu_link near the robot CoM, publishing calibrated angular velocities and accelerations to /imu. RGB-D Camera: 640x480 resolution at 30Hz on camera_link with optical frame transform, publishing color, depth, and PointCloud2.

Evaluator & Staff Viva Voce Q&A;

Q: Why did we change wheel collisions from STL meshes to cylinders?

A: STL mesh surfaces consist of faceted triangles whose edges snag on the ground plane in Gazebo physics, causing artificial bounce. Cylinders have mathematically smooth surfaces, giving continuous rolling contact.

Q: Why is lateral friction mu2 set to 0.1 on the wheels?

A: Because this is a 4-wheel skid-steer robot. When it turns, the tires must physically slide across the ground. A low lateral friction allows smooth skidding without excessive motor torque spikes.

Code #11: amr_world.sdf

Relative Workspace Path: src/assem2_robot/worlds/amr_world.sdf

Role in System:	Gazebo Harmonic Physics World & Testing Arena
Primary Tool / Node:	gz-sim8 (Physics, Sensors, SceneBroadcaster, Imu)
Topics & Signals:	Outputs: /world/amr_world/clock, sensor raycasts

Functional Purpose: amr_world.sdf

Configures the Gazebo Harmonic virtual testing arena. Specifies a fixed 5ms (200Hz) ODE physics time step, loads the Ogre 2.x sensor engine, and constructs an enclosed 10m x 10m warehouse arena with static obstacle pillars, boxes, and partition walls to evaluate SLAM mapping and dynamic navigation.

Full Source Code: amr_world.sdf

```
<?xml version="1.0" ?>
<sdf version="1.8">
  <world name="amr_world">
    <physics name="5ms" type="ignored">
      <max_step_size>0.005</max_step_size>
      <real_time_factor>1.0</real_time_factor>
    </physics>

    <!-- Essential Gazebo Harmonic System Plugins -->
    <plugin filename="gz-sim-physics-system" name="gz::sim::systems::Physics" />
    <plugin filename="gz-sim-user-commands-system" name="gz::sim::systems::UserCommands" />
    <plugin filename="gz-sim-scene-broadcaster-system" name="gz::sim::systems::SceneBroadcaster
  " />
    <plugin filename="gz-sim-sensors-system" name="gz::sim::systems::Sensors">
      <render_engine>ogre2</render_engine>
    </plugin>
    <plugin filename="gz-sim-imu-system" name="gz::sim::systems::Imu" />

    <!-- Ambient & Sun Lighting -->
    <light type="directional" name="sun">
      <cast_shadows>true</cast_shadows>
      <pose>0 0 10 0 0 0</pose>
      <diffuse>0.85 0.85 0.85 1</diffuse>
      <specular>0.2 0.2 0.2 1</specular>
      <attenuation>
        <range>1000</range>
        <constant>0.9</constant>
        <linear>0.01</linear>
        <quadratic>0.001</quadratic>
      </attenuation>
      <direction>-0.5 0.1 -0.9</direction>
    </light>

    <!-- Ground Plane -->
    <model name="ground_plane">
      <static>true</static>
      <link name="link">
        <collision name="collision">
          <geometry>
            <plane>
              <normal>0 0 1</normal>
              <size>50 50</size>
            </plane>
          </geometry>
          <surface>
            <friction>
              <ode>
                <mu>1.0</mu>
                <mu2>1.0</mu2>
              </ode>
            </friction>
          </surface>
        </collision>
        <visual name="visual">
          <geometry>
            <plane>
              <normal>0 0 1</normal>
              <size>50 50</size>
            </plane>
          </geometry>
          <material>
            <ambient>0.7 0.75 0.8 1</ambient>
            <diffuse>0.7 0.75 0.8 1</diffuse>
            <specular>0.2 0.2 0.2 1</specular>
          </material>
        </visual>
      </link>
    </model>

    <!-- Arena Boundary Walls (10m x 10m enclosure) -->
    <model name="wall_north">
      <static>true</static>
      <pose>0 5 0.5 0 0 0</pose>
      <link name="link">
        <collision name="c">
          <geometry><box><size>10.2 0.2 1.0</size></box></geometry>
        </collision>
        <visual name="v">
          <geometry><box><size>10.2 0.2 1.0</size></box></geometry>
```

```
          <material>
            <ambient>0.3 0.3 0.35 1</ambient>
            <diffuse>0.4 0.4 0.45 1</diffuse>
          </material>
        </visual>
      </link>
    </model>

    <model name="wall_south">
      <static>true</static>
      <pose>0 -5 0.5 0 0 0</pose>
      <link name="link">
        <collision name="c">
          <geometry><box><size>10.2 0.2 1.0</size></box></geometry>
        </collision>
        <visual name="v">
          <geometry><box><size>10.2 0.2 1.0</size></box></geometry>
          <material>
            <ambient>0.3 0.3 0.35 1</ambient>
            <diffuse>0.4 0.4 0.45 1</diffuse>
          </material>
        </visual>
      </link>
    </model>

    <model name="wall_east">
      <static>true</static>
      <pose>5 0 0.5 0 0 0</pose>
      <link name="link">
        <collision name="c">
          <geometry><box><size>0.2 10.0 1.0</size></box></geometry>
        </collision>
        <visual name="v">
          <geometry><box><size>0.2 10.0 1.0</size></box></geometry>
          <material>
            <ambient>0.3 0.3 0.35 1</ambient>
            <diffuse>0.4 0.4 0.45 1</diffuse>
          </material>
        </visual>
      </link>
    </model>

    <model name="wall_west">
      <static>true</static>
      <pose>-5 0 0.5 0 0 0</pose>
      <link name="link">
        <collision name="c">
          <geometry><box><size>0.2 10.0 1.0</size></box></geometry>
        </collision>
        <visual name="v">
          <geometry><box><size>0.2 10.0 1.0</size></box></geometry>
          <material>
            <ambient>0.3 0.3 0.35 1</ambient>
            <diffuse>0.4 0.4 0.45 1</diffuse>
          </material>
        </visual>
      </link>
    </model>

    <!-- Obstacles / Pillars for LiDAR Scan & Navigation -->
    <model name="pillar_1">
      <static>true</static>
      <pose>2.0 2.0 0.5 0 0 0</pose>
      <link name="link">
        <collision name="c">
          <geometry><cylinder><radius>0.4</radius><length>1.0</length></cylinder></geometry>
        </collision>
        <visual name="v">
          <geometry><cylinder><radius>0.4</radius><length>1.0</length></cylinder></geometry>
          <material>
            <ambient>0.8 0.3 0.2 1</ambient>
            <diffuse>0.8 0.3 0.2 1</diffuse>
          </material>
        </visual>
      </link>
    </model>

    <model name="pillar_2">
      <static>true</static>
      <pose>-2.0 -2.0 0.5 0 0 0</pose>
      <link name="link">
```

```
160 |         <collision name="c">
161 |           <geometry><cylinder><radius>0.35</radius><length>1.0</length></cylinder></geometry>
162 |         </collision>
163 |         <visual name="v">
164 |           <geometry><cylinder><radius>0.35</radius><length>1.0</length></cylinder></geometry>
165 |           <material>
166 |             <ambient>0.2 0.6 0.8 1</ambient>
167 |             <diffuse>0.2 0.6 0.8 1</diffuse>
168 |           </material>
169 |         </visual>
170 |       </link>
171 |     </model>
172 |
173 |     <model name="box_obstacle_1">
174 |       <static>true</static>
175 |       <pose>2.5 -2.0 0.4 0 0 0.4</pose>
176 |       <link name="link">
177 |         <collision name="c">
178 |           <geometry><box><size>0.8 1.2 0.8</size></box></geometry>
179 |         </collision>
180 |         <visual name="v">
181 |           <geometry><box><size>0.8 1.2 0.8</size></box></geometry>
182 |           <material>
183 |             <ambient>0.85 0.65 0.15 1</ambient>
184 |             <diffuse>0.85 0.65 0.15 1</diffuse>
185 |           </material>
186 |         </visual>
187 |       </link>
188 |     </model>
189 |
190 |     <model name="box_obstacle_2">
191 |       <static>true</static>
192 |       <pose>-2.5 2.0 0.4 0 0 -0.5</pose>
193 |       <link name="link">
194 |         <collision name="c">
195 |           <geometry><box><size>1.0 1.0 0.8</size></box></geometry>
196 |         </collision>
197 |         <visual name="v">
198 |           <geometry><box><size>1.0 1.0 0.8</size></box></geometry>
199 |           <material>
200 |             <ambient>0.3 0.75 0.4 1</ambient>
201 |             <diffuse>0.3 0.75 0.4 1</diffuse>
202 |           </material>
203 |         </visual>
204 |       </link>
205 |     </model>
206 |
207 |     <model name="barrier_center">
208 |       <static>true</static>
209 |       <pose>0.0 2.8 0.35 0 0 0</pose>
210 |       <link name="link">
211 |         <collision name="c">
212 |           <geometry><box><size>2.0 0.3 0.7</size></box></geometry>
213 |         </collision>
214 |         <visual name="v">
215 |           <geometry><box><size>2.0 0.3 0.7</size></box></geometry>
216 |           <material>
217 |             <ambient>0.5 0.3 0.7 1</ambient>
218 |             <diffuse>0.5 0.3 0.7 1</diffuse>
219 |           </material>
220 |         </visual>
221 |       </link>
222 |     </model>
223 |
224 |     <!-- AgileX Scout Mini AMR Robot Model -->
225 |     <include>
226 |       <uri>package://assem2_robot/urdf/Assem2.SLDASM.urdf</uri>
227 |       <name>assem2_robot</name>
228 |       <pose>0 0 0.02 0 0 0</pose>
229 |     </include>
230 |
231 |   </world>
232 | </sdf>
```

How the Code Works (Line-by-Line & Engineering Logic): amr_world.sdf

- **1. Physics Solver Configuration (Lines 1-8):**

max_step_size: 0.005 s (5 ms = 200 Hz) and real_time_factor: 1.0. A 5 ms step size balances high contact stability for fast-spinning wheels with efficient 1.0x real-time simulation on standard CPUs.

- **2. Essential System Plugins (Lines 9-17):**

Loads gz-sim-physics-system, gz-sim-sensors-system (with render_engine ogre2 for GPU LiDAR and camera), gz-sim-scene-broadcaster-system, and gz-sim-imu-system.

- **3. 10m x 10m Arena Walls & Obstacles (Lines 69-233):**

Four boundary walls prevent the robot from escaping sensor bounds. Interior obstacles include two 0.7m-diameter pillars, two large storage crates, and a central partition barrier that force Nav2 to plan non-trivial detour paths.

Evaluator & Staff Viva Voce Q&A;

Q: Why do we need ogre2 rendering engine in Gazebo Harmonic?

A: Ogre 2.x is required by Gazebo Harmonic's gpu_lidar and camera sensor plugins to perform hardware-accelerated raycasting and depth rendering on modern GPUs.

Code #12: amr_world_map.yaml

Relative Workspace Path: src/assem2_robot/maps/amr_world_map.yaml

Role in System:	2D Occupancy Grid Map Metadata Descriptor
Primary Tool / Node:	nav2_map_server
Topics & Signals:	Publishes: /map, /map_metadata

Functional Purpose: amr_world_map.yaml

The metadata descriptor file accompanying amr_world_map.pgm. Informs the Nav2 Map Server of the map's metric resolution (5cm/pixel), origin coordinates in world space, and probability thresholds for obstacle classification.

Full Source Code: amr_world_map.yaml

```
1 | image: amr_world_map.pgm
2 | mode: trinary
3 | resolution: 0.050
4 | origin: [-3.754, -5.021, 0]
5 | negate: 0
6 | occupied_thresh: 0.65
7 | free_thresh: 0.196
```

How the Code Works (Line-by-Line & Engineering Logic): amr_world_map.yaml

- **1. Resolution & Origin (Lines 3-4):**

resolution: 0.05 (each pixel is 5cm). origin: [-5.5, -5.5, 0.0] offsets the map so that pixel (0,0) corresponds to the bottom-left corner of the world, placing (0,0) right at the center of the 10m arena.

- **2. Classification Thresholds (Lines 5-6):**

occupied_thresh: 0.65 (probabilities >= 65% are obstacles); free_thresh: 0.25 (probabilities <= 25% are free driveable space).

Evaluator & Staff Viva Voce Q&A;

Q: How does AMCL use this file?

A: The Map Server publishes this grid to AMCL, which evaluates the likelihood of real-time LiDAR beams striking pre-mapped walls to update particle filter weights.

Code #13: CMakeLists.txt

Relative Workspace Path: src/assem2_robot/CMakeLists.txt

Role in System:	ROS 2 Ament CMake Build Specification
Primary Tool / Node:	colcon build (ament_cmake)
Topics & Signals:	N/A (Build Tool)

Functional Purpose: CMakeLists.txt

Build script that instructs colcon how to package our robot files. Installs config, launch, maps, meshes, urdf, and worlds directories to the package share folder, and installs patrol_mission.py to lib/assem2_robot as an executable.

Full Source Code: CMakeLists.txt

```
 1 | cmake_minimum_required(VERSION 3.8)
 2 | project(assem2_robot)
 3 |
 4 | if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
 5 |   add_compile_options(-Wall -Wextra -Wpedantic)
 6 | endif()
 7 |
 8 | find_package(ament_cmake REQUIRED)
 9 |
10 | install(
11 |   DIRECTORY config launch maps meshes urdf worlds
12 |   DESTINATION share/${PROJECT_NAME}
13 | )
14 |
15 | install(
16 |   PROGRAMS scripts/patrol_mission.py
17 |   DESTINATION lib/${PROJECT_NAME}
18 |   RENAME patrol_mission
19 | )
20 |
21 | install(
22 |   PROGRAMS scripts/patrol_mission.py
23 |   DESTINATION lib/${PROJECT_NAME}
24 | )
25 |
26 | ament_package()
```

How the Code Works (Line-by-Line & Engineering Logic): CMakeLists.txt

- **1. Directory Installation (Lines 10-13):**

Installs config, launch, maps, meshes, urdf, and worlds to share/assem2_robot. This allows ROS 2 launch files to find assets using get_package_share_directory('assem2_robot').

- **2. Python Script Executable (Lines 15-24):**

Installs scripts/patrol_mission.py into lib/assem2_robot/ and creates an alias named 'patrol_mission', enabling execution via: 'ros2 run assem2_robot patrol_mission'.

Evaluator & Staff Viva Voce Q&A;

Q: Why do we install files to share/ and lib/ in ROS 2?

A: In the modern ament build system, share/ holds immutable assets (launch, URDF, configs) while lib/<package_name>/ holds executable binaries and scripts.

Code #14: package.xml

Relative Workspace Path: src/assem2_robot/package.xml

Role in System:	ROS 2 Package Dependency Manifest (Format 3)
Primary Tool / Node:	rosdep / ament
Topics & Signals:	N/A (Package Management)

Functional Purpose: package.xml

Declares the package name, version, license, and explicit runtime execution dependencies (exec_depend) for robot_state_publisher, rviz2, ros_gz_sim, ros_gz_bridge, robot_localization, slam_toolbox, and nav2_bringup.

Full Source Code: package.xml

```
 1 | <?xml version="1.0"?>
 2 | <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.
   |   w3.org/2001/XMLSchema"?>
 3 | <package format="3">
 4 |   <name>assem2_robot</name>
 5 |   <version>1.0.0</version>
 6 |   <description>URDF and simulation package for four-wheeler robot</description>
 7 |   <maintainer email="todo@email.com">todo</maintainer>
 8 |   <license>BSD</license>
 9 |
10 |   <buildtool_depend>ament_cmake</buildtool_depend>
11 |
12 |   <exec_depend>robot_state_publisher</exec_depend>
13 |   <exec_depend>joint_state_publisher</exec_depend>
14 |   <exec_depend>rviz2</exec_depend>
15 |   <exec_depend>ros_gz_sim</exec_depend>
16 |   <exec_depend>ros_gz_bridge</exec_depend>
17 |   <exec_depend>robot_localization</exec_depend>
18 |   <exec_depend>slam_toolbox</exec_depend>
19 |   <exec_depend>nav2_bringup</exec_depend>
20 |
21 |   <export>
22 |     <build_type>ament_cmake</build_type>
23 |   </export>
24 | </package>
```

How the Code Works (Line-by-Line & Engineering Logic): package.xml

- **1. Package Identification (Lines 1-9):**

Declares format='3', name='assem2_robot', version='1.0.0', and BSD license.

- **2. Execution Dependencies (Lines 12-20):**

Lists all 8 runtime software dependencies so that 'rosdep install --from-paths src' can automatically verify and install every required library on a clean Ubuntu 24.04 system.

Evaluator & Staff Viva Voce Q&A;

Q: What happens if an exec_depend is missing from package.xml?

A: The package might build, but running launch files on a clean machine will fail with PackageNotFoundError when it tries to import the missing node.